Lógica y Representación 1

Programación Orientada a Objetos (POO)

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen





PROBLEMA

ALGORITMO



1. Entender el problema
2. Diseñar un plan
3. Implementar el plan
4. Revisar el plan

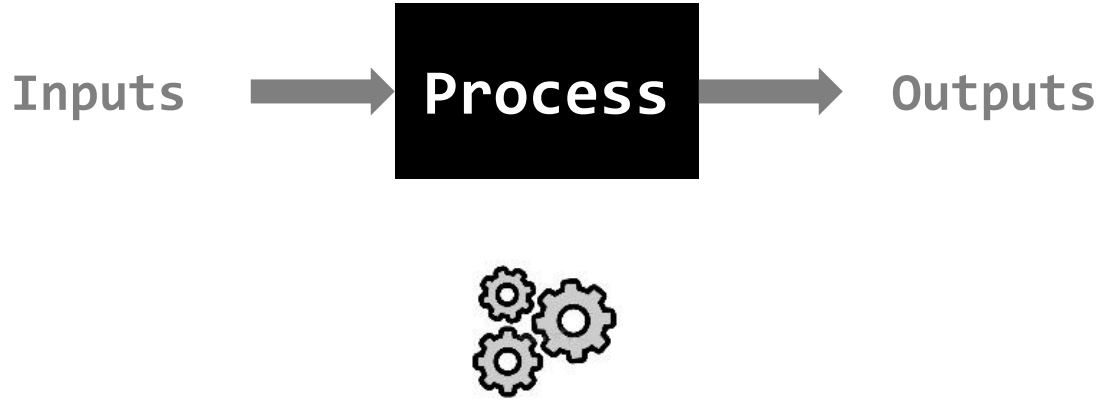
Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**







Repaso: Sistema de administración de notas de un curso

- Un curso admite máximo **`N = 10`** estudiantes. De cada uno se guarda su **Id (texto)**, su **nombre** y su **nota**.
- Por ahora hay 4 inscritos; se pueden agregar más hasta llenar el cupo.
- Se necesita un programa para administrar el curso: agregar, eliminar, actualizar, buscar, listar y **ordenar**.

Id	Nombre	Nota
"000"	"Ana"	3.0
"123"	"Beto"	4.5
"987"	"Carla"	2.0
"007"	"Dario"	4.0
	(libre)	
	...	
	(libre)	

4 inscritos

6 cupos libres (hasta N = 10)

Repaso: Sistema de administración de notas de un curso

- El programa muestra un menú y repite hasta que se elige **0. Salir**.
- Cada opción es una función que recibe los vectores y cantidad, y usa lo visto en clases anteriores.
- Cada opción fue una función que recibía los vectores y cantidad.
- El programa completo está en el repositorio.

Salida tentativa en consola

```
=== Administración del curso ===
1. Agregar estudiante
2. Eliminar estudiante
3. Actualizar nota
4. Buscar estudiante
5. Listar curso
6. Ordenar por nota
7. Ordenar por nombre
0. Salir

Opción:
```

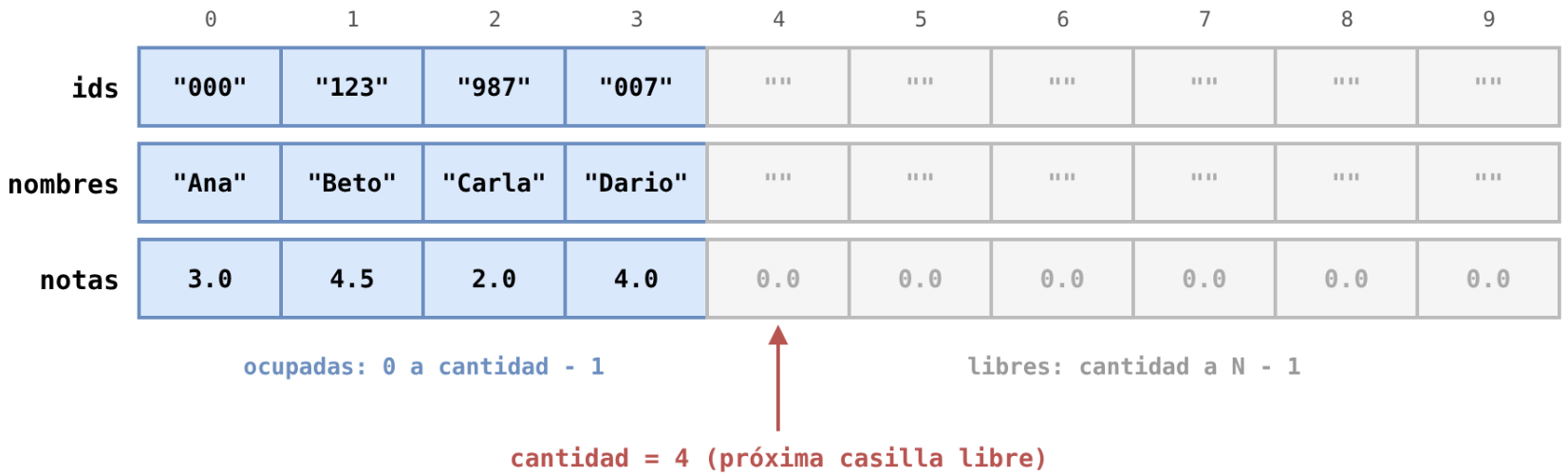
Repaso: Sistema de administración de notas de un curso

- **Menú de siete opciones**: agregar, eliminar, actualizar nota, buscar, listar, ordenar por nota y ordenar por nombre.

Opción	Qué hace	Usa
1. Agregar	Agrega al final si hay cupo y el Id no existe	cantidad, búsqueda lineal
2. Eliminar	Quita un estudiante por Id y corre los siguientes a la izquierda	búsqueda lineal, desplazamiento
3. Actualizar nota	Cambia la nota de un Id	búsqueda lineal
4. Buscar	Muestra los datos de un Id	búsqueda lineal
5. Listar	Muestra los **`cantidad`** inscritos	recorrido
6. Ordenar por nota	De mayor a menor	selección + intercambio
7. Ordenar por nombre	Alfabético	selección + intercambio

Repaso: Sistema de administración de notas de un curso

- **Problema**: administrar las notas de un curso de máximo `N = 10` estudiantes. De cada uno se guarda su **Id** (texto), su **nombre** y su **nota**.
- Se resolvió con tres vectores paralelos: **`ids`**, **`nombres`** y **`notas`**. Un estudiante ocupa la misma posición en los tres.
- **`cantidad`** dice cuántas casillas están ocupadas; el resto está libre.

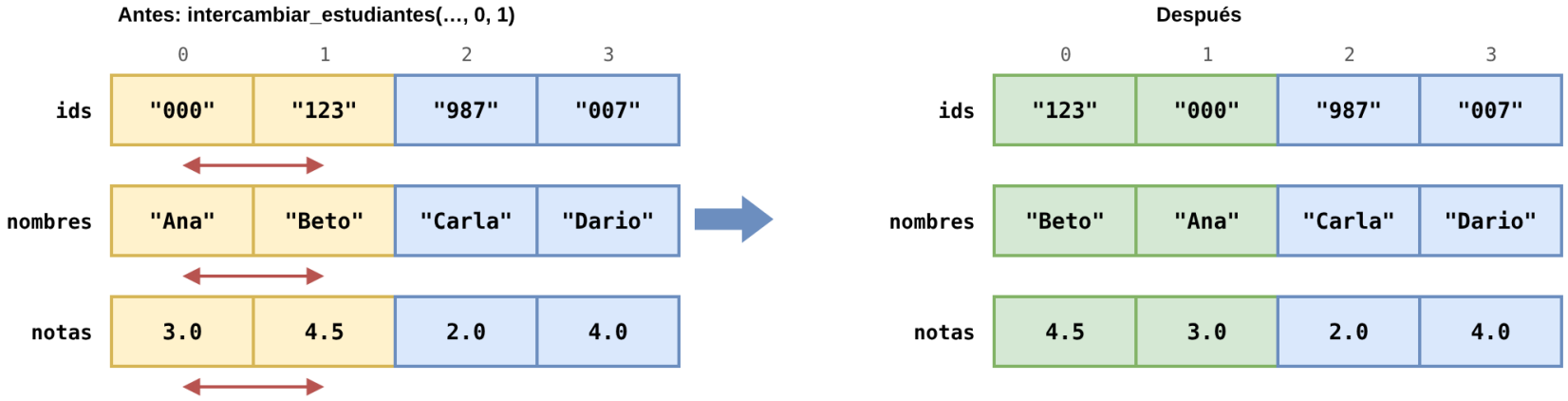


Cada vector tiene N = 10 casillas; solo las primeras 4 (cantidad) tienen datos.

Link simulación

Repaso: intercambiar en los tres vectores

- Para ordenar, cada intercambio tiene que mover al estudiante completo: su Id, su nombre y su nota.
- Por eso **`intercambiar_estudiantes`** llama tres veces a **`intercambiar`**, una por vector y con las mismas posiciones.
- Si se olvida uno de los tres vectores, los datos de los estudiantes quedan mezclados y Python no avisa.



El estudiante de la posición 1 pasa a la 0 (y viceversa) en los TRES vectores: cada Id y nombre sigue unido a su nota.

```
def intercambiar_estudiantes(ids, nombres, notas, i, j):
    intercambiar(ids, i, j)
    intercambiar(nombres, i, j)
    intercambiar(notas, i, j)
```

Link simulación

El problema pendiente: ¿qué fue incómodo?

- Con arreglos paralelos el programa funciona, pero deja estas incomodidades.
- La POO promete resolverlas. Se comprueba una por una, con el mismo problema, los mismos datos y el mismo menú.

#	Con arreglos paralelos	Con objetos	Estado
1	Un estudiante no existe	Existe: es un Estudiante	por comprobar
2	Cada intercambio son tres	Se intercambia un Estudiante	por comprobar
3	cantidad va a todas partes	Vive dentro de Curso	por comprobar
4	Una función por criterio	Sigue igual	por comprobar
+	¿Y si se agrega el correo?	Solo cambia Estudiante	por comprobar

No es un problema solo de este curso: CS50 (Harvard) llega al mismo punto con una agenda guardada en dos arreglos paralelos, nombres y teléfonos (semana 3, **Structs**).

Plan de la clase

Bloque 1 - Del problema al lenguaje

- Qué es un objeto y qué es una clase.
- Del diagrama UML a Python con cuatro ejemplos: un rectángulo, un ventilador, una cuenta bancaria y un arreglo de cuentas.

Bloque 2 - El curso con objetos

- **`Estudiante`** y **`Curso`**: del diagrama UML al código.
- El mismo menú, ahora con objetos.
- Comprobar el tablero, una incomodidad a la vez.

En Python, todo dato es un objeto

- **En Python todos los datos son objetos**: una lista, un texto, un número.
- Cada objeto tiene métodos (operaciones o funciones) propias, que se usan con la forma **`objeto.metodo()`**: **`notas.append(4.0)`** agrega al final de la lista; **`nombre.upper()`** pasa el texto a mayúsculas.
- **`type()`** muestra de qué clase es cada objeto.

Lo nuevo: crear clases propias, como **`Estudiante`**.

Código

```
notas = [3.0, 4.5, 2.0]
print(notas)
notas.append(4.0)
print(notas)
nombre = "Ana"
print(nombre)
print(nombre.upper())
print(type(notas))
print(type(nombre))
print(type(4.0))
```

Link simulación

Salida

```
[3.0, 4.5, 2.0]
[3.0, 4.5, 2.0, 4.0]
Ana
ANA
<class 'list'>
<class 'str'>
<class 'float'>
```

Paradigmas de programación

Un paradigma es una forma de ver el problema y de organizar su solución en un lenguaje de programación.

Paradigma	Idea central	Lenguajes
Programación estructurada	La solución se arma con tres estructuras de control: secuencia, selección e iteración, organizadas en funciones.	C, Pascal, Fortran
Programación funcional	La solución se descompone en funciones, como en matemáticas, que se pueden llamar a sí mismas.	Haskell, Lisp, Erlang, Elixir
Programación orientada a objetos (POO)	El mundo es un conjunto de objetos, con datos y comportamiento, que interactúan para resolver el problema.	Java, C++, C#, Smalltalk
Programación reactiva	El programa reacciona a un flujo de datos que cambia con el tiempo.	Elm; bibliotecas como RxJS (JavaScript) y RxJava (Java)

Hasta ahora: programación estructurada. En la programación estructurada, los datos están separados de las funciones y viajan de una a otra como parámetros. En la POO, los datos y sus **métodos** viajan juntos, dentro del objeto.

Un paradigma es una forma de pensar el problema

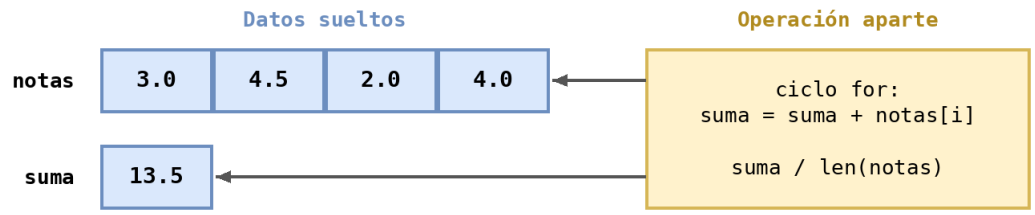
- Un paradigma no es un lenguaje ni una instrucción: es **la forma de pensar el problema antes de escribir código**. Es qué se pregunta uno primero.
- **Una analogía cotidiana: atender un restaurante.** El problema es el mismo (dar de comer a 20 personas), pero hay distintas formas de organizarlo:

Paradigma	**La pregunta de partida**	**En el restaurante**
Estructurada	¿Qué **pasos** se siguen, y en qué orden?	**Una receta**: primero esto, luego aquello; si falta sal, se agrega; se repite para cada plato.
Funcional	¿Qué **transformaciones** se aplican?	**Estaciones**: lo crudo sale picado, lo picado sale cocido, lo cocido sale servido.
Orientada a objetos	¿**Quiénes** participan, y qué sabe hacer cada uno?	**Roles**: el mesero toma pedidos, el cocinero cocina, la caja cobra.
Reactiva	¿A qué **eventos** se reacciona?	**Pedidos**: la cocina espera; cada pedido que llega dispara el trabajo.

El mismo problema, dos paradigmas

Problema: Calcular el promedio de las notas (3.0, 4.5, 2.0 y 4.0) de un curso. Las dos versiones dan 3.375.

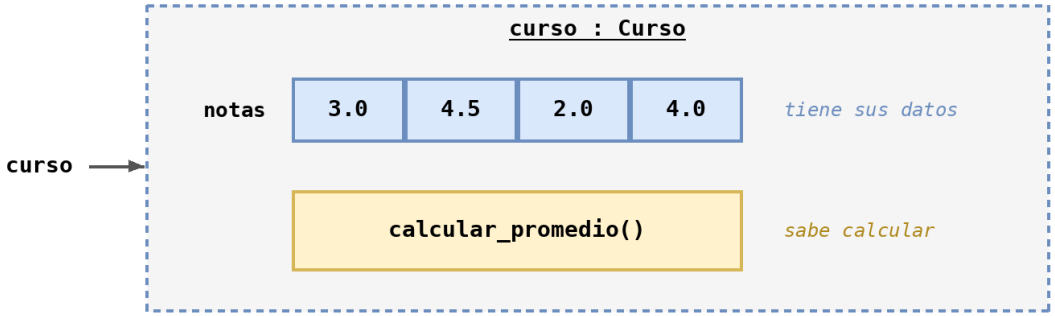
Programación estructurada




```
notas = [3.0, 4.5, 2.0, 4.0]
suma = 0.0
for i in range(len(notas)):
    suma = suma + notas[i]
print(suma / len(notas))
```

Link simulación

Programación Orientada a Objetos (POO)




```
class Curso:
    def __init__(self, notas):
        self.notas = notas

    def calcular_promedio(self):
        suma = 0.0
        for i in range(len(self.notas)):
            suma = suma + self.notas[i]
        return suma / len(self.notas)

curso = Curso([3.0, 4.5, 2.0, 4.0])
print(curso.calcular_promedio())
```

Link simulación

¿Cómo se guardan juntos datos de distinto tipo?

Un estudiante tiene un **Id** (texto), un **nombre** (texto) y una **nota** (real). Un arreglo no sirve para guardarlos juntos: sus datos son todos del mismo tipo.

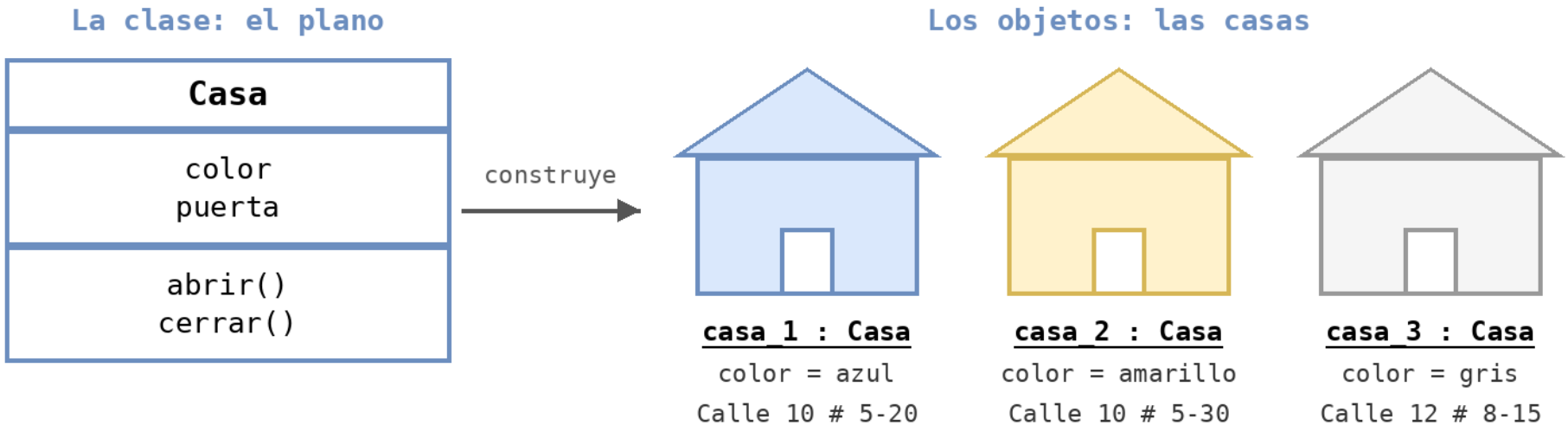
Estructura	**Tipos de datos**	**Cada dato se identifica por**	**Ejemplo**
Arreglo	Uno solo	Su posición	`notas[2]`
Registro (struct en C, record en Pascal)	Varios	Su nombre	`ana.nota`
Clase (POO)	Varios	Su nombre, y además tiene **métodos propios**.	`ana.set_nota(4.0)`

Python permite listas con tipos mezclados, como `["000", "Ana", 3.0]`; también JavaScript, Ruby y PHP. En C, Java o Pascal, un arreglo es de un solo tipo. Aun en Python, esa lista identifica cada dato por su posición: `ana[2]` no dice que es la nota. En este curso, las listas se usan como arreglos: un solo tipo.

Una clase es como un registro que además sabe hacer cosas: guarda los datos con nombre y **los métodos** que los usan.

Clase y objeto: el plano y las casas

- **Una clase es como el plano de una casa**: dice qué tiene cada casa (color, puerta) y qué se puede hacer con ella (abrir, cerrar).
- **Un objeto es cada casa construida con ese plano**: todas tienen la misma forma, pero cada una tiene sus propios valores: una es azul, otra amarilla.
- **Con un plano se construyen muchas casas**: de una clase se crean muchos objetos, también llamados instancias.

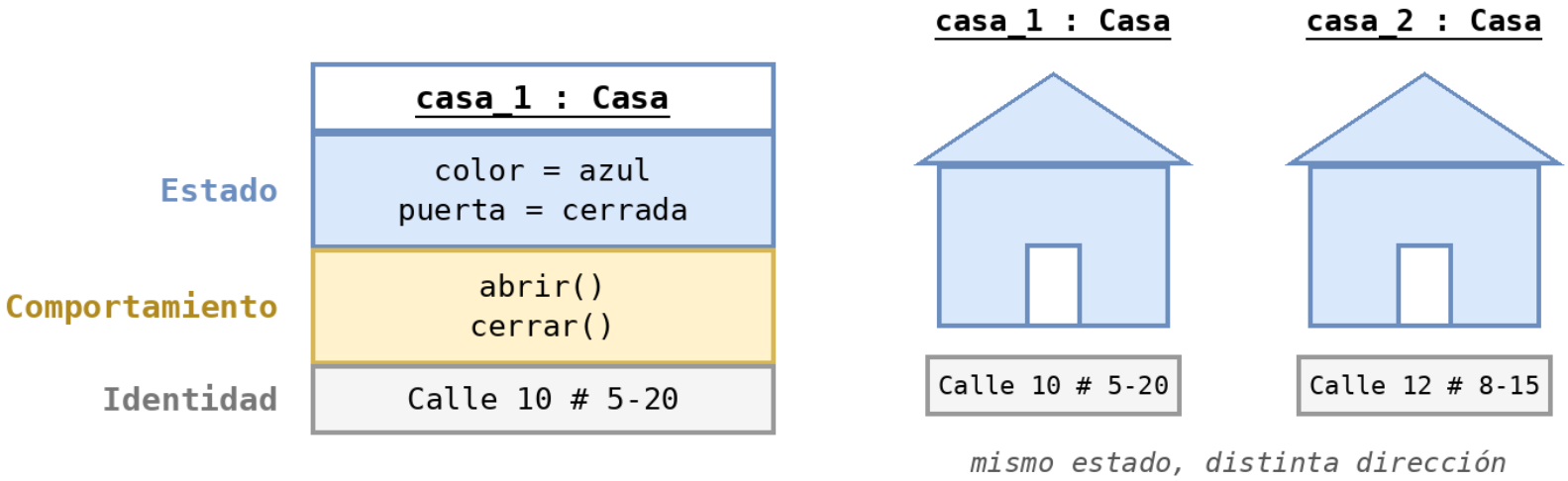


Un plano, muchas casas: cada una con sus propios valores y su propia dirección.

Estado, comportamiento e identidad

Todo objeto tiene tres características. En las casas del plano se ven así:

Un objeto tiene	Qué es	En las casas
Estado	Los valores de sus datos en un momento dado	**`casa_1`**: color azul, puerta cerrada. Si se pinta o se abre la puerta, su estado cambia.
Comportamiento	Lo que sabe hacer	Abrir y cerrar la puerta
Identidad	Es uno solo, distinto de cualquier otro, aunque tenga los mismos datos	La dirección: dos casas idénticas siguen siendo dos casas

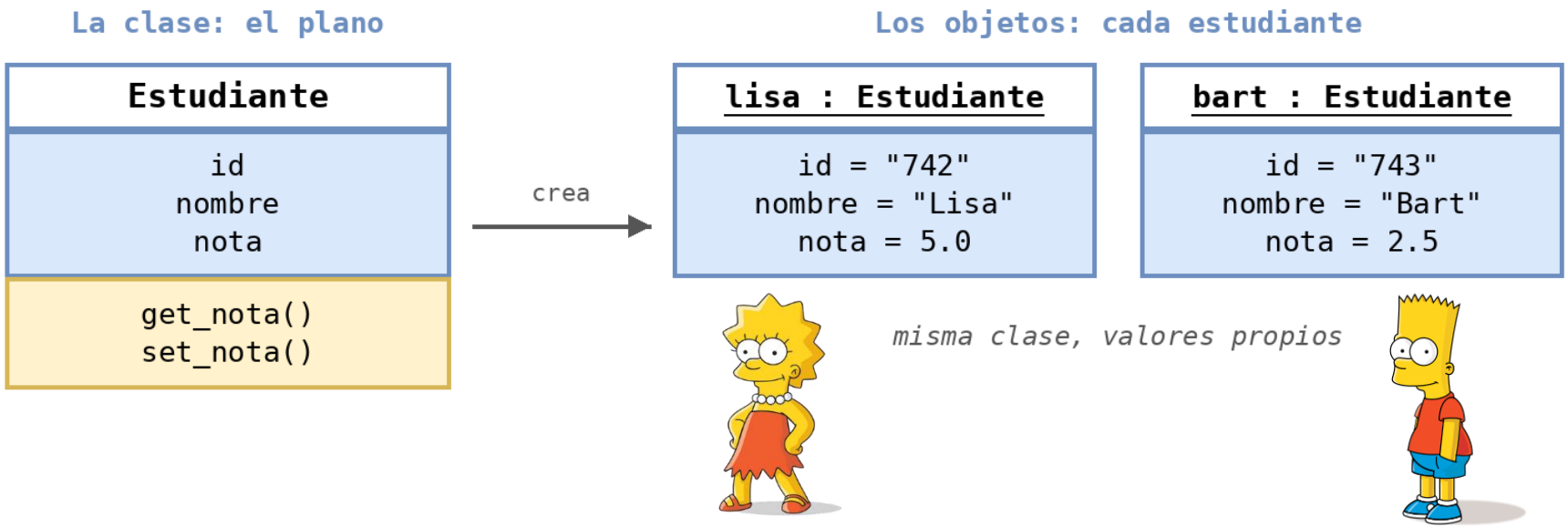


Dos casas idénticas, del mismo color y con la puerta cerrada, tienen el mismo estado, pero siguen siendo dos casas: cada una tiene su propia dirección.

Del plano de la casa al estudiante

Lo mismo pero con los estudiantes de un curso:

	En las casas	**En los estudiantes**
Clase (el plano)	Casa	Estudiante
Objetos	`casa_1, casa_2`	`lisa, bart`
Estado	Color azul, puerta cerrada	Id "742", nombre "Lisa", nota 5.0
Comportamiento	Abrir y cerrar la puerta	Consultar y cambiar la nota: `get_nota()`, `set_nota()`
Identidad	Su dirección: dos casas idénticas siguen siendo dos casas	Si Bart copia las notas de Lisa, siguen siendo dos estudiantes



De aquí en adelante, las clases y los objetos se representan así, en UML: sin dibujos, solo con cajas.

¿Qué permite la POO?

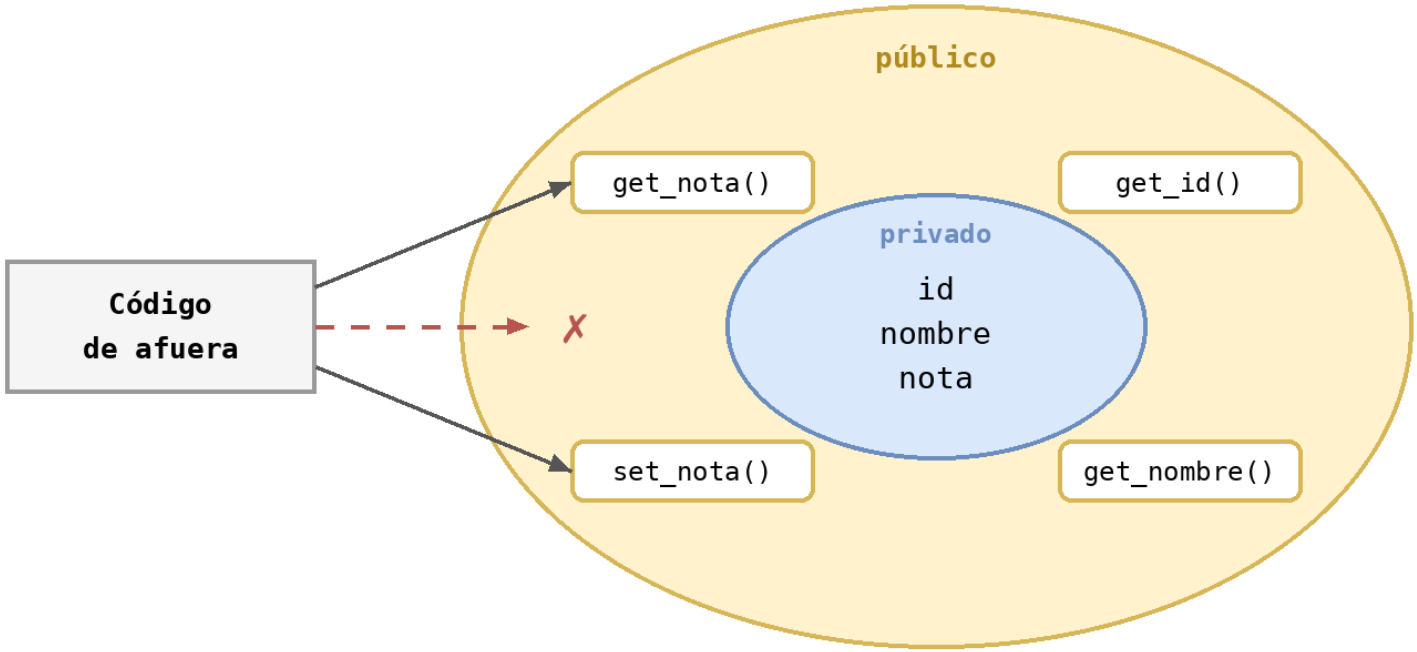
La POO tiene cuatro propiedades que la distinguen.

Propiedad	Qué permite	En las casas	En los estudiantes
Abstracción	Quedarse solo con lo que importa para el problema	Del plano interesan el color y la puerta, no cada ladrillo	Se guardan su Id, su nombre y su nota, no su estatura
Encapsulamiento	Juntar los datos con sus métodos y proteger los datos	La puerta se abre con **`abrir()`**; nadie desarma la cerradura desde la calle	La nota solo se cambia con **`set_nota()`**
Herencia	Crear una clase a partir de otra	Una casa con piscina es una casa, con algo más	Un estudiante de posgrado es un estudiante, con algo más: su tesis
Polimorfismo	Un mismo método, con comportamiento distinto según el objeto	**`abrir()`** en una casa y en un garaje automático	**`calcular_nota_final()`** en pregrado y en posgrado

En este curso solo se ven la abstracción y el encapsulamiento. La herencia y el polimorfismo salen del alcance de este curso y corresponden a cursos posteriores.

Encapsulamiento: los datos se protegen

- El objeto guarda sus datos en el centro: son **privados**.
- Alrededor están sus métodos: son **públicos**.
- El código de afuera **no toca los datos directamente** (flecha roja): **le pide a un método lo que necesita** (flechas grises). Para consultar la nota usa **`get_nota()`**; para cambiarla, **`set_nota()`**.

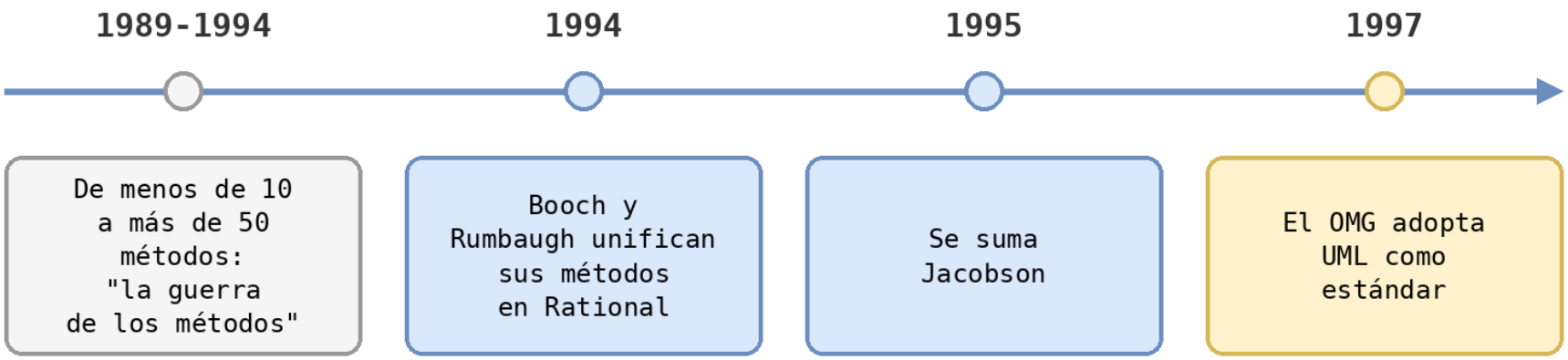


Desde afuera solo se usan los métodos (público); los datos quedan protegidos (privado).

Como el sistema de notas de la universidad: nadie entra a la base de datos a escribir una nota. Se consulta y se cambia a través del sistema, que revisa que la nota sea válida.

¿Por qué UML?

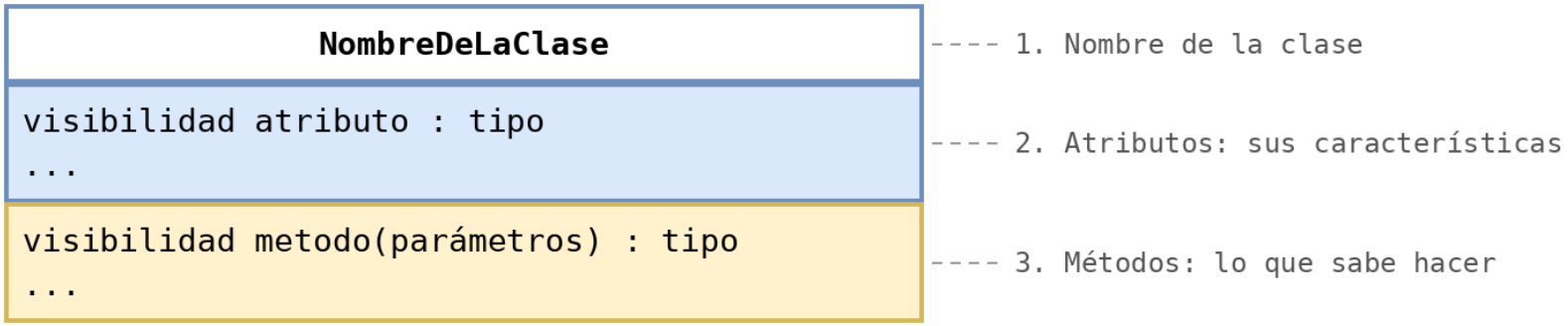
- Entre 1989 y 1994, los métodos de modelado orientado a objetos pasaron de menos de 10 a **más de 50**. Cada equipo dibujaba distinto, y se habló de **“la guerra de los métodos”**.
- En 1994, en la empresa Rational, **Grady Booch** y **James Rumbaugh** empezaron a unificar sus métodos; en 1995 se sumó **Ivar Jacobson**.
- En **1997**, el **OMG** (**Object Management Group**) adoptó UML como estándar. **UML (Lenguaje Unificado de Modelado)** es el estándar para dibujar un programa orientado a objetos antes de escribirlo.



Para profundizar: Kobryn, C. (1999). UML 2001: A Standardization Odyssey. Communications of the ACM, 42(10). https://cacm.acm.org/research/uml-2001/

UML: la plantilla de una clase

- En UML, cada clase se dibuja como una caja de tres partes:
  - **Nombre**: el nombre de la clase.
  - **Atributos**: sus características.
  - **Métodos**: lo que sabe hacer.



Elemento	Se escribe	Ejemplo
Atributo	`visibilidad nombre : tipo`	`- nota : float`
Constructor	`visibilidad NombreDeLaClase(parámetros)`	`+ Estudiante(id: str, nombre: str, nota: float)`
Método	`visibilidad nombre(parámetros) : tipo de retorno`	`+ set_nota(nota: float) : bool`

Visibilidad: **+** público, **-** privado. Se verá después.

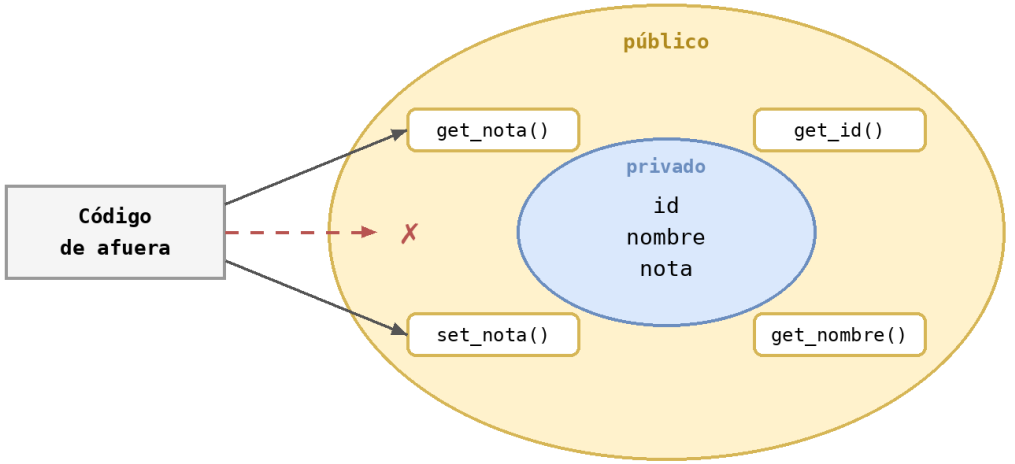
Especificadores de visibilidad

- En el diagrama de clases UML, el símbolo (**-**, **+**, **#**, **~**) al inicio de cada atributo o método dice quién puede usarlo:

Símbolo	Visibilidad	Quién puede usarlo	En Python
+	Público	Cualquier código	Nombre normal: base
-	Privado	Solo la propia clase	Dos guiones bajos: **__nota**
#	Protegido	La clase y las que heredan de ella	Un guion bajo: **_nota** (solo convención)
~	Paquete	Las clases del mismo paquete	No existe en Python

Python no obliga a respetar la visibilidad: `__nota` esconde el nombre del atributo y **`_nota`** es solo un aviso para otros programadores.

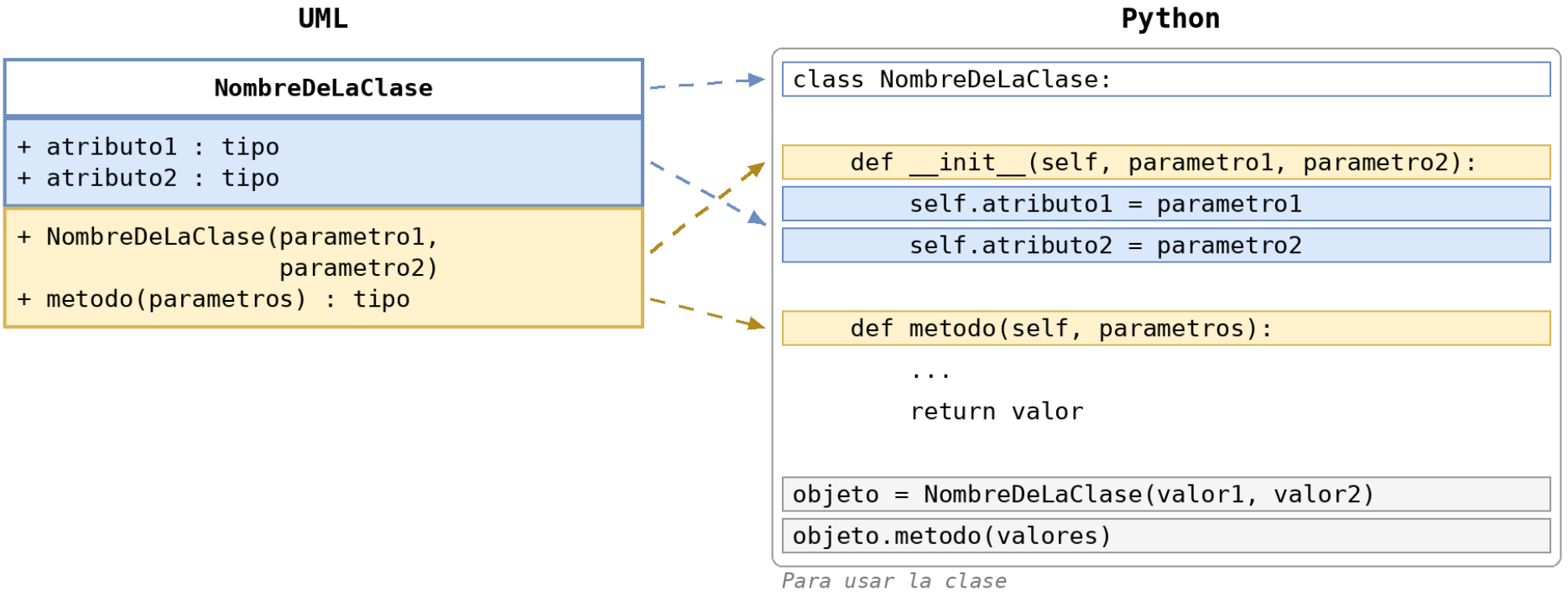
En este curso se usan + para los métodos y - para los atributos.



Desde afuera solo se usan los métodos (público); los datos quedan protegidos (privado).

De UML a Python

- **Cada parte del diagrama tiene su lugar en el código**: el **nombre** en **`class`**, los **atributos** y el **constructor** en **`__init__`**, y cada **método** en su **`def`**.
- El **constructor** se escribe **`__init__`**, pero se usa con el nombre de la clase: **`objeto = NombreDeLaClase(valor1, valor2)`**.
- **Los atributos no se declaran aparte**: nacen dentro de **`__init__`**, con **`self`**.
- **Todo método** recibe **`self`** como primer parámetro: es el objeto que se está usando. En UML, **`self`** no aparece.



Resumen: de UML a Python

Esta tabla resume cómo se escribe en Python cada parte del diagrama UML. Lo que más cambia: el constructor se llama `__init__`, y todo método recibe `self` como primer parámetro.

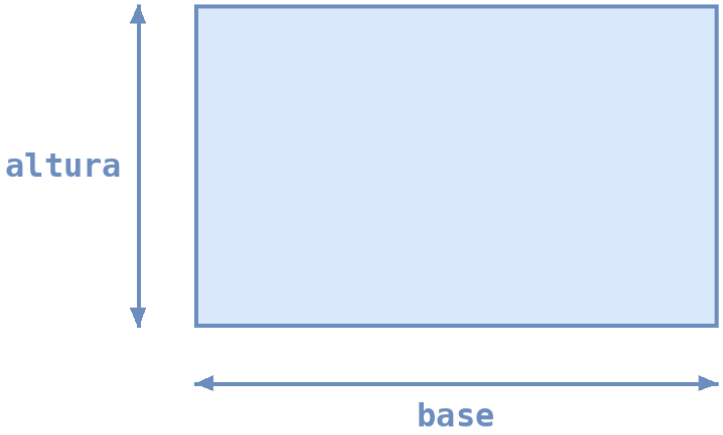
Elemento	En UML	En Python
Clase	`NombreDeLaClase`	`class NombreDeLaClase:`
Constructor	`+ NombreDeLaClase(parametros)`	`def __init__(self, parametros):`
Atributo	`+ atributo : tipo`	`self.atributo = parametro` (dentro de `__init__`)
Método	`+ metodo(parametros) : tipo`	`def metodo(self, parametros):`
Crear un objeto	–	`objeto = NombreDeLaClase(valores)`
Usar un método	–	`objeto.metodo(valores)`

Las dos últimas filas no están en el diagrama de clases: sirven para instanciar (crear) un objeto y usarlo.

Ejemplo 1: Clase `Rectangulo`

Enunciado: Diseñe una clase llamada **`Rectangulo`** para representar un rectángulo. La clase contiene:

- Dos atributos, **`base`** y **`altura`**.
- Un **constructor** que crea un rectángulo con la **`base`** y la **`altura`** dadas.
- Un método **`calcular_area()`** que retorna el área.
- Un método **`calcular_ perimetro()`** que retorna el perímetro.



área = base × altura

perímetro = 2 × base + 2 × altura

Ej.: base 3.0 y altura 4.0 → área 12.0, perímetro 14.0

Dibuje el diagrama UML y luego impleméntela. Pruébela con dos rectángulos: de **`3.0 × 4.0`** y de **`5.0 × 2.0`**.

Ejemplo 1: Clase `Rectangulo`

Enunciado: Diseñe una clase llamada **`Rectangulo`** para representar un rectángulo.

El enunciado

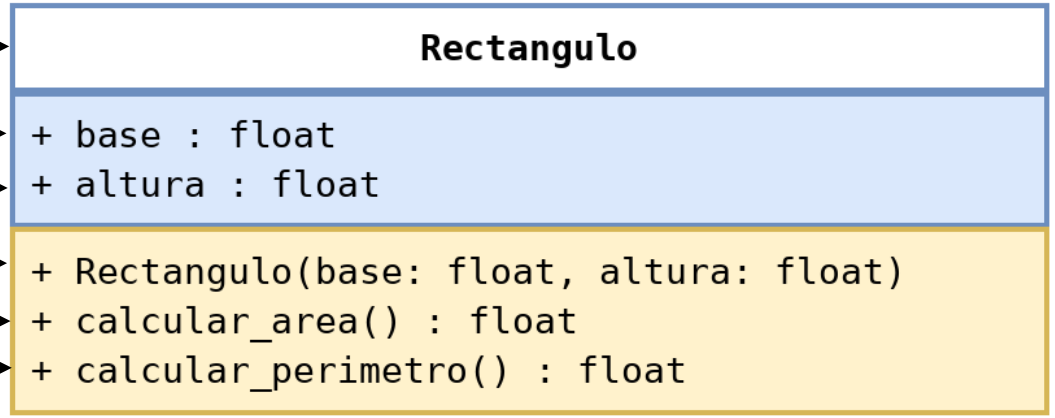
Clase: `Rectangulo`

Atributos:
`base`
`altura`

Constructor

Métodos:
`calcular_area()`
`calcular_perimetro()`

Diagrama UML

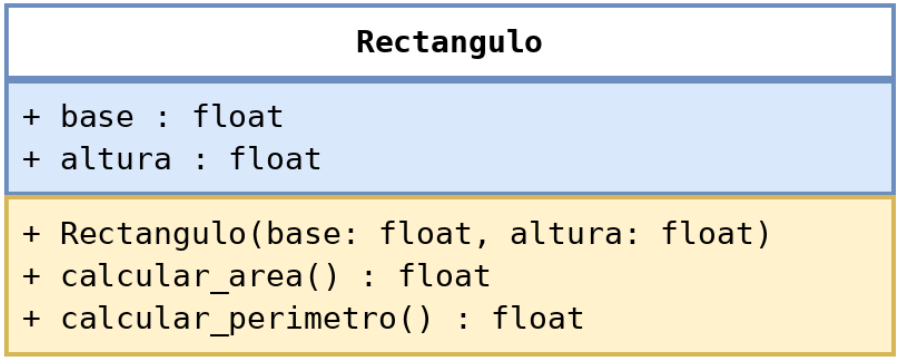


El enunciado no dice los tipos: se deciden al diseñar. La **`base`** y la **`altura`** pueden tener decimales: **`float`**.

Ejemplo 1: del UML a Python

Enunciado: Diseñe una clase llamada **`Rectangulo`** para representar un rectángulo.

Diagrama UML



Implementación en Python

```
class Rectangulo:
   def __init__(self, base, altura):
     self.base = base
     self.altura = altura

   def calcular_area(self):
     return self.base * self.altura

   def calcular_perimetro(self):
     return 2 * self.base + 2 * self.altura
```

Es la plantilla aplicada: el nombre en **`class`**, los atributos dentro de **`__init__`** con **`self`**, y cada método en su **`def`**, con **`self`** como primer parámetro.

Ejemplo 1: instanciar y usar un objeto

Instanciar: crear un objeto (una instancia) a partir de una clase, como construir una casa a partir de su plano.

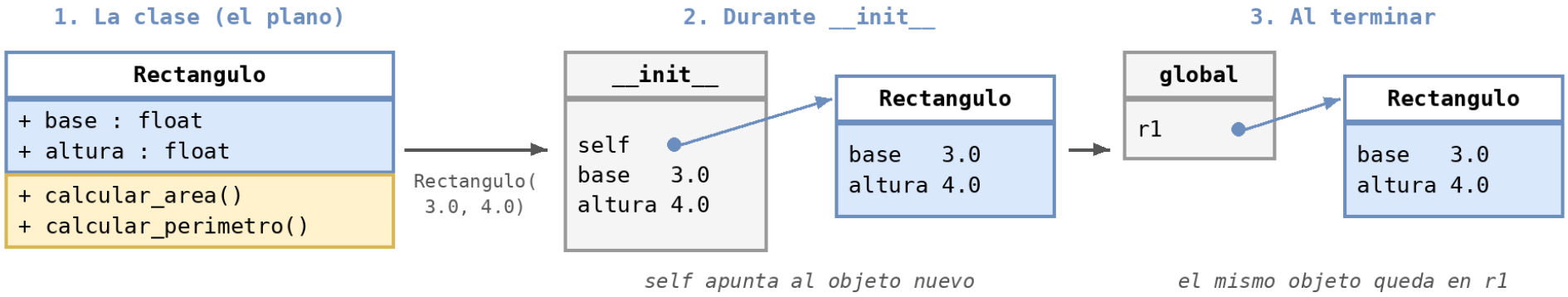
Código Python

```
r1 = Rectangulo(3.0, 4.0)
print(r1.calcular_area())
print(r1.calcular_perimetro())
```

Link simulación

Salida

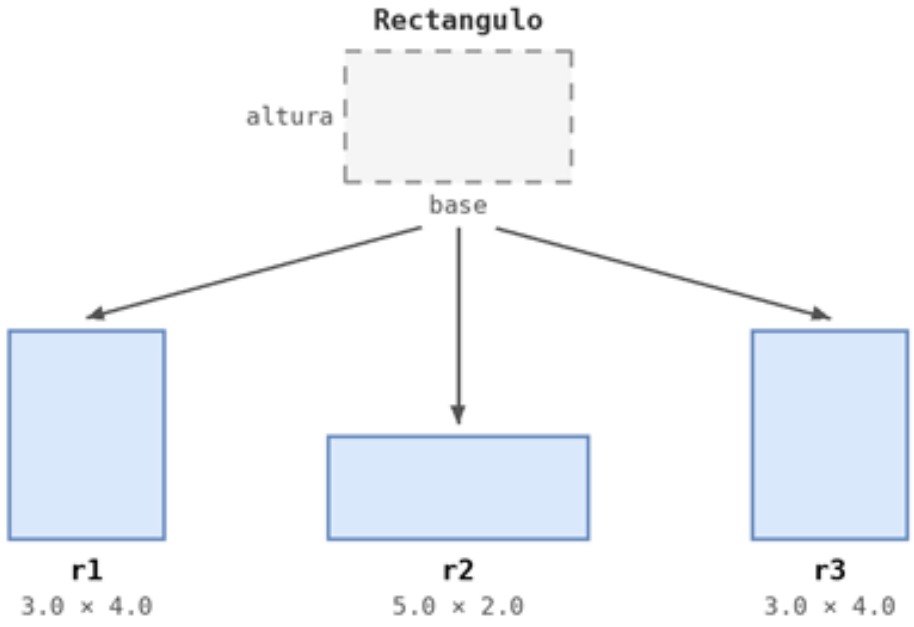
```
12.0
14.0
```



`self` es el objeto que se está construyendo o usando. Por eso, en **`r1.calcular_area()`**, **`self.base`** es la base de **`r1`**.

Ejemplo 1: Objetos y pruebas

De una **clase** se crean **muchos objetos**, cada uno con sus propios valores.



Código Python

```
r1 = Rectangulo(3.0, 4.0)
r2 = Rectangulo(5.0, 2.0)
assert r1.calcular_area() == 12.0
assert r2.calcular_area() == 10.0
assert r1.calcular_perimetro() == 14.0
assert r2.calcular_perimetro() == 14.0

r3 = Rectangulo(3.0, 4.0)
print(r1 is r3)
print("Rectangulo: todas las pruebas pasaron")
```

Link simulación

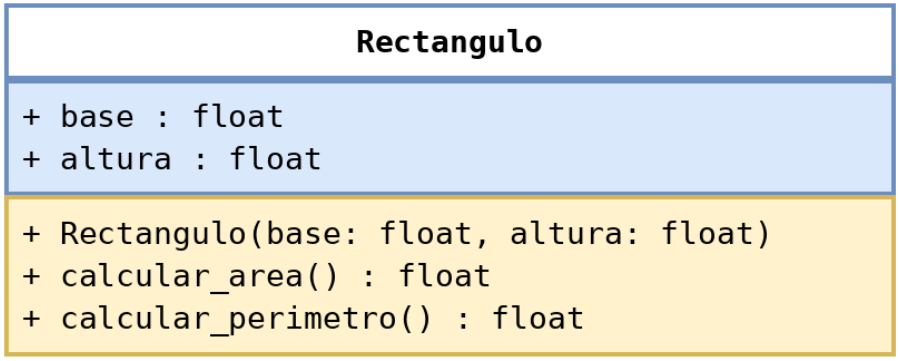
Salida

```
False
Rectangulo: todas las pruebas pasaron
```

`r1` y **`r3`** tienen los mismos valores, pero son dos objetos distintos: **`is`** compara la identidad. Como las dos casas idénticas, cada una en su dirección.

Ejemplo 1: ¿y si alguien cambia la base?

Los atributos son públicos (+): desde fuera de la clase cualquiera puede cambiarlos, sin ningún control.



Código Python

```
r1 = Rectangulo(3.0, 4.0)
print(r1.calcular_area())
r1.base = -5.0
print(r1.calcular_area())
print(r1.calcular_perimetro())
```

Link simulación

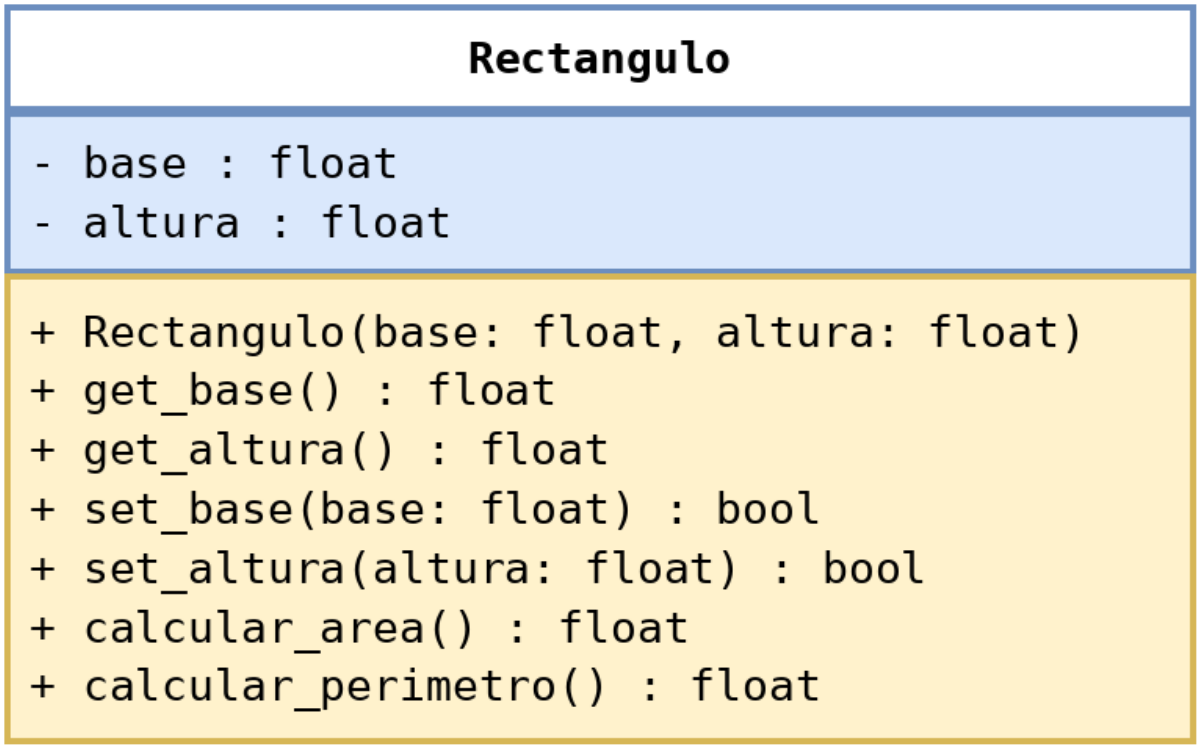
Salida

```
12.0
-20.0
-2.0
```

Un rectángulo con base negativa no existe, pero el programa lo aceptó y siguió calculando.
¿Cómo hacemos que el objeto proteja sus propios datos?

Ejemplo 1 mejorado: atributos privados

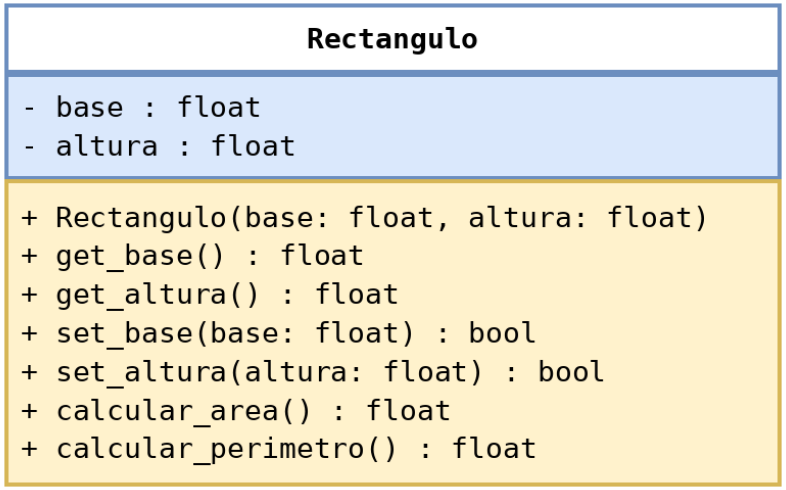
Privado (-): solo los métodos de la clase pueden leer o cambiar el atributo. Desde fuera se usan métodos: **`get_`** para consultar y **`set_`** para cambiar, validando.



En esta implementación **`set_base`** y **`set_altura`** devuelven **`True`** si aceptaron el valor y **`False`** si lo rechazaron. En la industria, en vez de devolver **`False`**, se lanza un **error** (una **excepción**).

Ejemplo 1 mejorado: del UML a Python

A continuación se muestra la implementación en Python del diagrama UML mejorado para la clase **`Rectangulo`**:



En Python, un atributo es privado si su nombre empieza con dos guiones bajos: **`self.__base`**

```
class Rectangulo:
    def __init__(self, base, altura):
        self.__base = base
        self.__altura = altura

    def get_base(self):
        return self.__base

    def get_altura(self):
        return self.__altura

    def set_base(self, base):
        cambiada = False
        if base > 0:
            self.__base = base
            cambiada = True
        return cambiada

    def set_altura(self, altura):
        cambiada = False
        if altura > 0:
            self.__altura = altura
            cambiada = True
        return cambiada

    def calcular_area(self):
        return self.__base * self.__altura

    def calcular_perimetro(self):
        return 2 * self.__base + 2 * self.__altura
```

Ejemplo 1 mejorado: pruebas

Se prueba un **caso típico** (`5.0`), uno **límite** (`0.0`) y uno **especial** (`-5.0`).

Código Python

```
r1 = Rectangulo(3.0, 4.0)
assert r1.set_base(5.0) == True
assert r1.calcular_area() == 20.0
assert r1.set_base(0.0) == False
assert r1.set_base(-5.0) == False
assert r1.get_base() == 5.0
print("Rectangulo: todas las pruebas pasaron")
```

Link simulación

Salida

```
Rectangulo: todas las pruebas pasaron
```

El valor rechazado no entra: después de intentar con 0.0 y -5.0, la base sigue en 5.0.

Observaciones importantes:

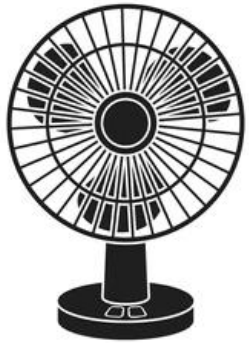
- En Python (link), `__` cambia el nombre del atributo por dentro: desde fuera, **`r1.__base`** no llega a la base verdadera. En Java (link), private es un candado; en Python, **`__`** es un letrero de "no pasar".
- Desde fuera de la clase se usan siempre **`get_`** y **`set_`**: es la única forma correcta de leer o cambiar un **atributo privado**.

Ejemplo 2: el ventilador

Diseñe una clase llamada Ventilador. La clase contiene:

- Tres constantes LENTO, MEDIO y RAPIDO, con los valores 1, 2 y 3, para indicar la velocidad.
- Cuatro atributos privados, cada uno con su valor por defecto:

Atributo	Tipo	Valor por defecto
`velocidad`	`int`	`LENTO`
`radio`	`float`	`5.0`
`color`	`str`	`"azul"`
`encendido`	`bool`	`False`

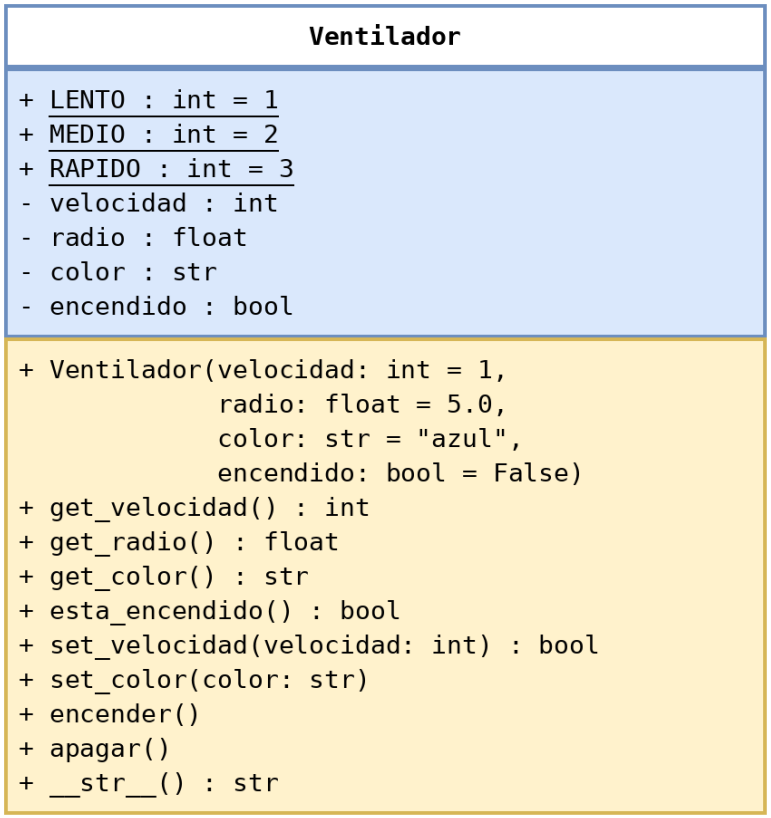


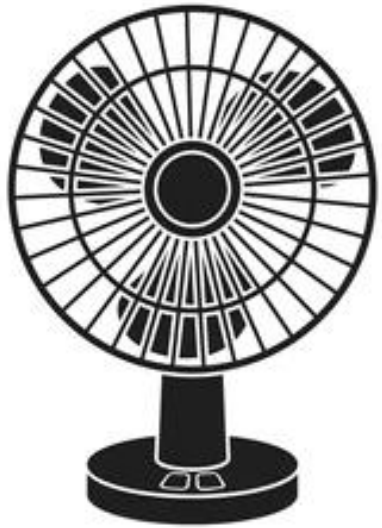
- Un constructor que crea un ventilador; si no se le dan valores, usa los valores por defecto.
- Métodos para consultar cada atributo.
- Un método para cambiar la velocidad, que solo acepta 1, 2 o 3, y otro para cambiar el color.
- Métodos para encender y apagar el ventilador.
- Un método que devuelve una descripción del ventilador en texto.

Dibuje el diagrama UML, implemente la clase y escriba pruebas con dos ventiladores: uno rápido, de **`radio 10.0`**, color "**`amarillo`**" y encendido; otro con los valores por defecto.

Ejemplo 2: del enunciado al UML

- **Subrayado**: el atributo es de la clase, no de cada objeto. Hay un solo `LENTO`, compartido por todos los ventiladores.
- **Sin tipo de retorno**: el método no devuelve nada, como el constructor.
- **Los parámetros pueden tener un valor por defecto**: `velocidad: int = 1`

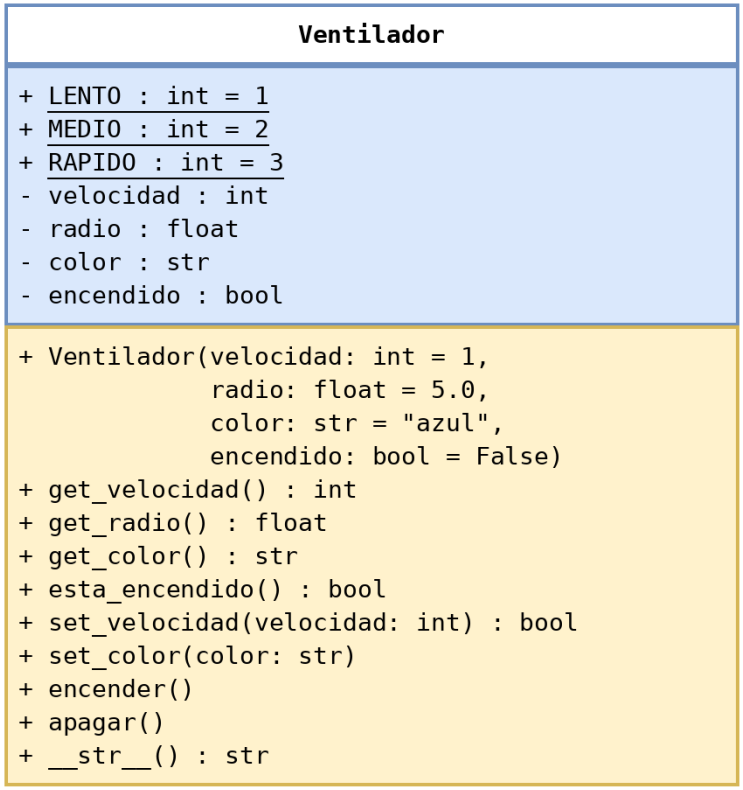




¿Por qué no hay `set_radio`? Una clase no necesita un setter por cada atributo: un ventilador no cambia de tamaño.

Ejemplo 2: del UML a Python (1/2)

Implementación en Python de la clase **Ventilador**



Código de la clase (Parte 1)

```
class Ventilador:
    LENTO = 1
    MEDIO = 2
    RAPIDO = 3

    def __init__(self, velocidad=1, radio=5.0,
                 color="azul", encendido=False):
        self.__velocidad = velocidad
        self.__radio = radio
        self.__color = color
        self.__encendido = encendido

    def get_velocidad(self):
        return self.__velocidad

    def get_radio(self):
        return self.__radio

    def get_color(self):
        return self.__color

    def esta_encendido(self):
        return self.__encendido
```

Creación de los objetos

```
v1 = Ventilador(Ventilador.RAPIDO, 10.0, "amarillo", True)
v2 = Ventilador()
print(v1.get_velocidad(), v1.get_radio(), v1.get_color(), v1.esta_encendido())
print(v2.get_velocidad(), v2.get_radio(), v2.get_color(), v2.esta_encendido())
```

Link simulación

`v2 = Ventilador()` usa todos los valores por defecto. En Java se escribirían varios constructores (sobrecarga); en Python basta uno con valores por defecto.

Ejemplo 2: del UML a Python (2/2)

Implementación en Python de la clase **Ventilador**

- **__str__** devuelve el texto que muestra **print(v2)**. Sin él, **print** mostraría algo como **<__main__.Ventilador object at 0x...>**.
- Los métodos sin tipo de retorno en el UML no llevan **return**: cambian el estado del objeto. En Python, devuelven **None**.

Creación de los objetos

```
v2 = Ventilador()
print(v2)
v2.set_velocidad(Ventilador.MEDIO)
print(v2.encender())
print(v2)
```

Link simulación

Código de la clase (Parte 2)

```
def set_velocidad(self, velocidad):
        cambiada = False
        if velocidad >= Ventilador.LENTO and
           velocidad <= Ventilador.RAPIDO:
            self.__velocidad = velocidad
            cambiada = True
        return cambiada

    def set_color(self, color):
        self.__color = color

    def encender(self):
        self.__encendido = True

    def apagar(self):
        self.__encendido = False

    def __str__(self):
        estado = "apagado"
        if self.__encendido:
           estado = "encendido"
           return ("Ventilador " + self.__color
                   + ", radio " + str(self.__radio)
                   + ", velocidad "
                   + str(self.__velocidad)
                   + ", " + estado)
```

encender() no devuelve nada: por eso **print(v2.encender())** muestra **None**. Lo útil es el cambio: el ventilador queda encendido.

Ejemplo 2: pruebas

`str(v2)` devuelve el mismo texto que muestra `print(v2)`: las dos usan **`__str__`**. Así se puede probar con **`assert`**.

Código Python

```
v2 = Ventilador()
assert v2.set_velocidad(4) == False
assert v2.set_velocidad(Ventilador.RAPIDO) == True
assert str(v2) == "Ventilador azul, radio 5.0, velocidad 3, apagado"
print("Ventilador: todas las pruebas pasaron")
```

Link simulación

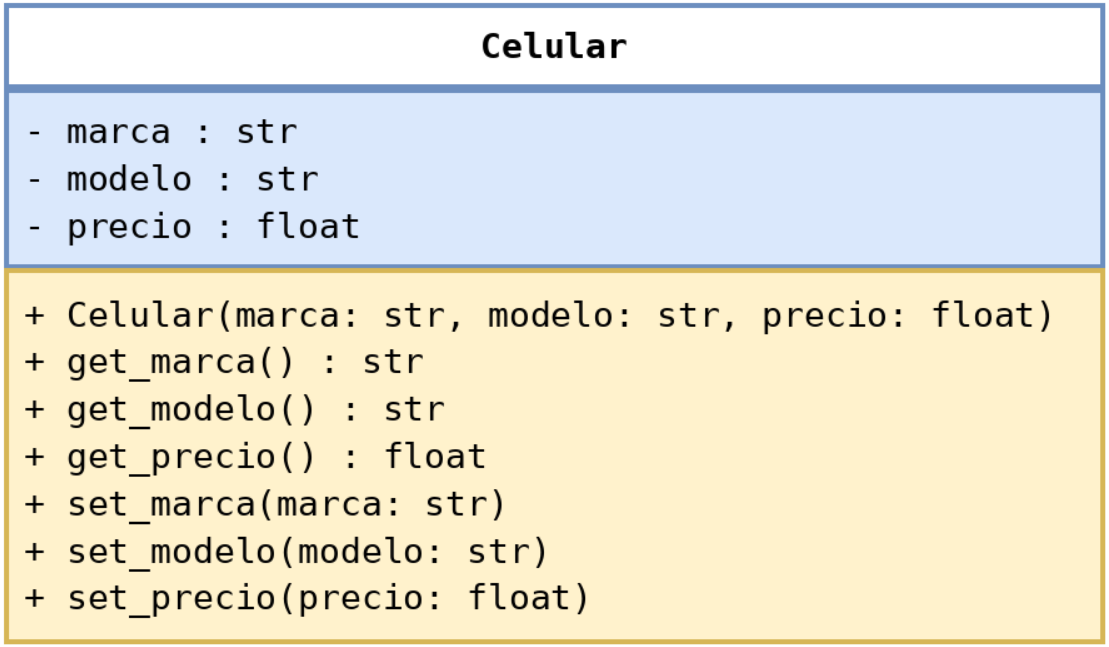
Salida

```
Ventilador: todas las pruebas pasaron
```

Se prueba el rechazo (**4** no es una velocidad válida), el cambio aceptado y que el texto refleje el nuevo estado: velocidad **3**.

Ejemplo 3: Hágalo usted mismo

Implemente en Python la clase Celular a partir de su diagrama UML.

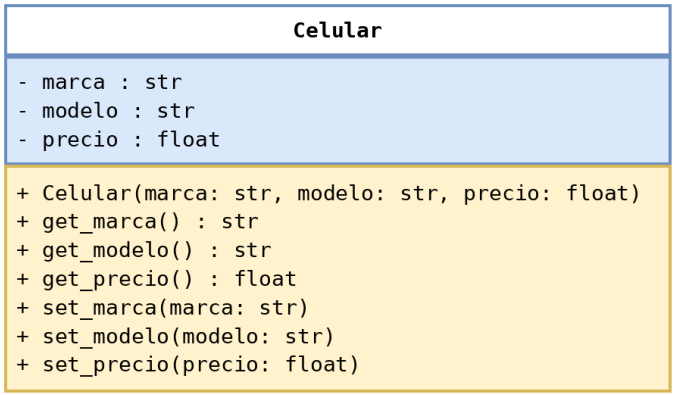




- **Pruebas**: cree un celular, cambie su precio y verifique los cambios con **`assert`**.
- **Reto**: haga que **`set_precio`** rechace los precios menores o iguales a **`0`** y devuelva **`True`** o **`False`**. ¿Qué cambia en el diagrama UML?

Ejemplo 3: solución

Implementación en Python de la clase **`Celular`**



Pruebas

```
c1 = Celular("Xiaomi", "Redmi 13", 650000.0)
assert c1.get_marca() == "Xiaomi"
assert c1.get_precio() == 650000.0
c1.set_precio(599000.0)
assert c1.get_precio() == 599000.0
print("Celular: todas las pruebas pasaron")
```

Salida

```
Celular: todas las pruebas pasaron
```

Código de la clase

```
class Celular:
    def __init__(self, marca, modelo, precio):
        self.__marca = marca
        self.__modelo = modelo
        self.__precio = precio

    def get_marca(self):
        return self.__marca

    def get_modelo(self):
        return self.__modelo

    def get_precio(self):
        return self.__precio

    def set_marca(self, marca):
        self.__marca = marca

    def set_modelo(self, modelo):
        self.__modelo = modelo

    def set_precio(self, precio):
        self.__precio = precio
```

Link simulación

Reto – Solución: ¿Qué cambia en el diagrama UML?

```
def set_precio(self, precio):
        cambiado = False
        if precio > 0:
            self.__precio = precio
            cambiado = True
        return cambiado
```

Ejemplo 4: la cuenta bancaria

Diseñe dos clases:

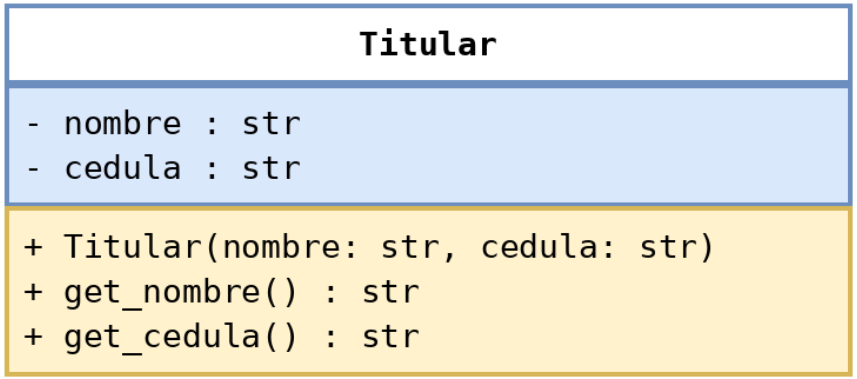
Clase `Titular`	**Clase `CuentaBancaria`**
La cual contiene: • Un atributo privado **nombre** (str). • Un atributo privado **cedula** (str). • Un constructor que crea un titular con su **nombre** y su **cédula**. • Métodos para consultar cada atributo.	La clase contiene: • Un atributo privado **titular**, de tipo **Titular**. • Un atributo privado **saldo** (float), con valor por defecto 0.0. • Un constructor que crea una cuenta con su **titular** y su **saldo** inicial. • Métodos para consultar el titular y el saldo. • Un método **depositar** que suma un valor al saldo. • Un método **retirar** que resta un valor del saldo, solo si el saldo alcanza; devuelve **True** o **False**. • Un método **transferir** que pasa un valor de esta cuenta a otra cuenta, solo si el saldo alcanza; devuelve **True** o **False**. • Un método que devuelve una descripción de la cuenta en texto.

Además:

- Dibuje el diagrama UML de cada clase y el que las relaciona.
- Pruebe las clases: Homero abre su cuenta con 30000.0 y Marge con el saldo por defecto; Homero le transfiere 15000.0 a Marge, y después Marge intenta transferir más de lo que tiene.

Ejemplo 4: la clase Titular

La clase **`Titular`** no tiene nada nuevo: atributos privados, constructor y métodos para consultar. Lo nuevo está en la cuenta, que guarda un Titular como atributo.




```
class Titular:
    def __init__(self, nombre, cedula):
        self.__nombre = nombre
        self.__cedula = cedula

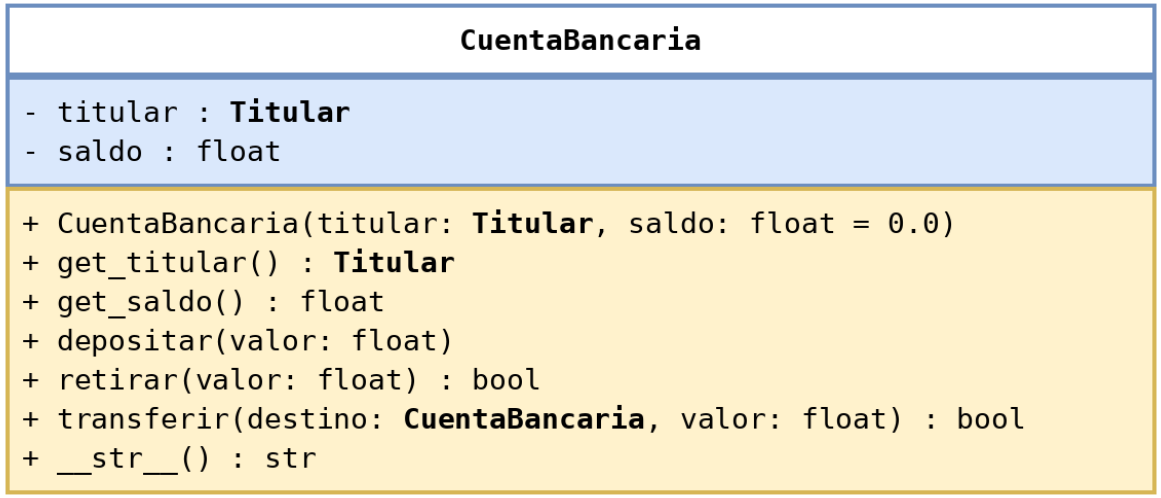
    def get_nombre(self):
        return self.__nombre

    def get_cedula(self):
        return self.__cedula
```

- **¿Y los setters? El enunciado solo pide consultar**: una cédula no cambia. No todo atributo necesita un setter.
- **¿Por qué la cédula es `str` y no `int`?** Porque no se hacen cuentas con ella y puede empezar con cero.
- **¿Por qué separar al titular de la cuenta?** Porque una persona puede tener varias cuentas: sus datos se escriben una sola vez.

Ejemplo 4: la clase CuentaBancaria

- **Un tipo puede ser una clase**: en este caso el objeto **`titular`** guarda un objeto **`Titular`** dentro de la cuenta.
- **Un parámetro también**: `transferir` recibe otra cuenta (`destino : CuentaBancaria`).



- **¿Y `set_saldo`? No existe**: en un banco nadie escribe el saldo a mano. Solo cambia con **`depositar`**, **`retirar`** y **`transferir`**.
- **¿Qué devuelve `get_titular()`?** Un objeto **`Titular`**, no un texto. Para obtener el nombre se puede usar: **`cuenta.get_titular().get_nombre()`**
- **¿Por qué `depositar` no lleva tipo de retorno?** Porque no devuelve nada; **`retirar`** sí: avisa con True o False si el saldo alcanzó.

Ejemplo 4: las dos clases relacionadas

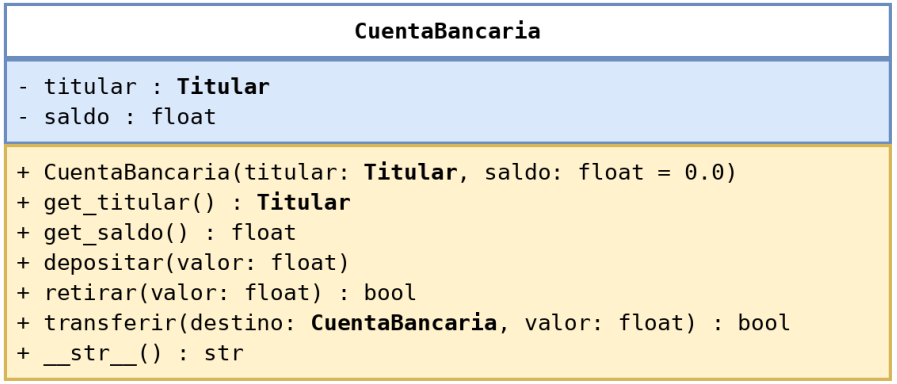
- **Asociación**: una línea entre dos clases que se relacionan. La palabra **`titular`**, junto a la flecha, es el nombre del atributo que guarda la relación.
- **La flecha indica quién conoce a quién**: la cuenta conoce a su titular; el titular no sabe qué cuentas tiene.
- **Multiplicidad**: cuántos objetos participan.
  - **`1`**: exactamente uno
  - **`1..*`**: uno o más.



- **¿Por qué la flecha va de la cuenta al titular?** Porque **`CuentaBancaria`** tiene el atributo **`titular`**; **`Titular`** no guarda cuentas.
- **¿Por qué las cajas no muestran atributos ni métodos?** Porque este diagrama muestra la relación. Los detalles de cada clase ya están en su propio UML.
- **Esto es solo un abrebocas**: diseñar diagramas UML para programas grandes se estudia en Ingeniería de Software. Aquí basta con la idea: cuando un programa crece, sus clases se relacionan, y UML permite dibujar esas relaciones.

Ejemplo 4: del UML a Python

Clase `CuentaBancaria`



- **¿Qué guarda `self.__titular`?** El objeto Titular completo, no solo el nombre. En Python Tutor se ve como una flecha de la cuenta al titular: la misma flecha del diagrama UML.
- **¿Por qué `__str__` escribe `self.__titular.get_nombre()`?** Porque el nombre es privado de Titular: la cuenta se lo pide con su método, como lo haría cualquiera desde fuera.
- **¿Qué hace `cuenta_h.get_titular().get_cedula()`?** Dos pasos: primero obtiene el Titular, y después le pide la cédula.

```
class CuentaBancaria:
    def __init__(self, titular, saldo=0.0):
        self.__titular = titular
        self.__saldo = saldo

    def get_titular(self):
        return self.__titular

    def get_saldo(self):
        return self.__saldo

    def depositar(self, valor):
        self.__saldo = self.__saldo + valor

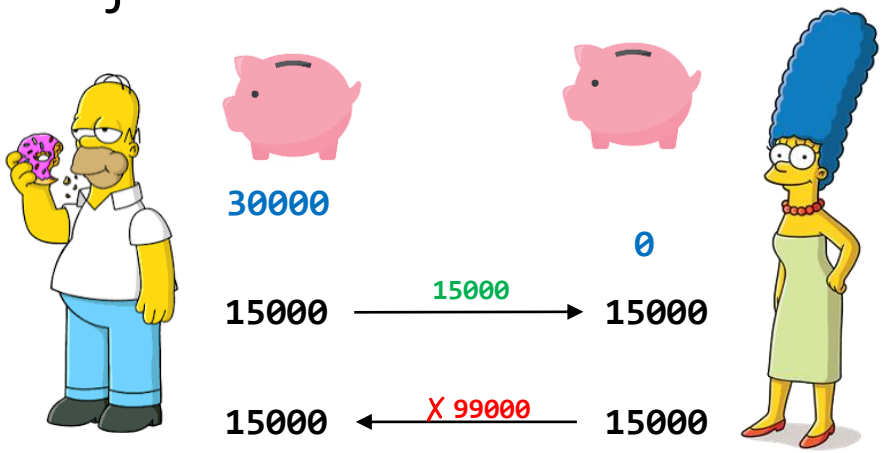
    def retirar(self, valor):
        retirado = False
        if valor <= self.__saldo:
            self.__saldo = self.__saldo - valor
            retirado = True
        return retirado

    def __str__(self):
        return (self.__titular.get_nombre()
                + ": $" + str(self.__saldo))

    def transferir(self, destino, valor):
        transferido = False
        if self.retirar(valor):
            destino.depositar(valor)
            transferido = True
        return transferido
```

Ejemplo 4: crear y usar los objetos

Recordemos el enunciado de la prueba:
Homero abre su cuenta con 30000.0 y Marge con el saldo por defecto; Homero le transfiere 15000.0 a Marge, y después Marge intenta transferir más de lo que tiene.



Código

```
# Homero abre su cuenta con 30000.0 y Marge con el saldo por defecto
homero = Titular("Homero Simpson", "1001")
marge = Titular("Marge Simpson", "1002")
cuenta_h = CuentaBancaria(homero, 30000.0)
cuenta_m = CuentaBancaria(marge)
print(cuenta_h)
print(cuenta_m)
# Homero le transfiere 15000.0 a Marge, y después Marge intenta transferir más de lo que tiene
print(cuenta_h.transferir(cuenta_m, 15000.0))
print(cuenta_m.transferir(cuenta_h, 99000.0))
print(cuenta_h)
print(cuenta_m)
```

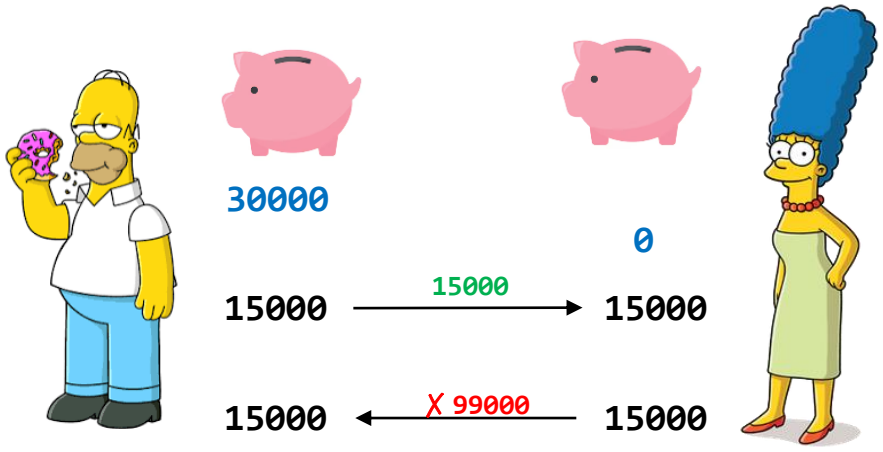
Link simulación

Salida

```
Homero Simpson: $30000.0
Marge Simpson: $0.0
True
False
Homero Simpson: $15000.0
Marge Simpson: $15000.0
```

Ejemplo 4: pruebas

Se prueba el escenario del enunciado: la transferencia aceptada, la rechazada, y que en cada caso los dos saldos queden bien.




```
homero = Titular("Homero Simpson", "1001")
marge = Titular("Marge Simpson", "1002")
cuenta_h = CuentaBancaria(homero, 30000.0)
cuenta_m = CuentaBancaria(marge)
assert cuenta_m.get_saldo() == 0.0
# Homero le transfiere 15000.0 a Marge
assert cuenta_h.transferir(cuenta_m, 15000.0) == True
assert cuenta_h.get_saldo() == 15000.0
assert cuenta_m.get_saldo() == 15000.0
# Marge intenta transferir más de lo que tiene
assert cuenta_m.transferir(cuenta_h, 99000.0) == False
assert cuenta_m.get_saldo() == 15000.0
assert cuenta_h.get_saldo() == 15000.0
print("CuentaBancaria: todas las pruebas pasaron")
```

Link simulación

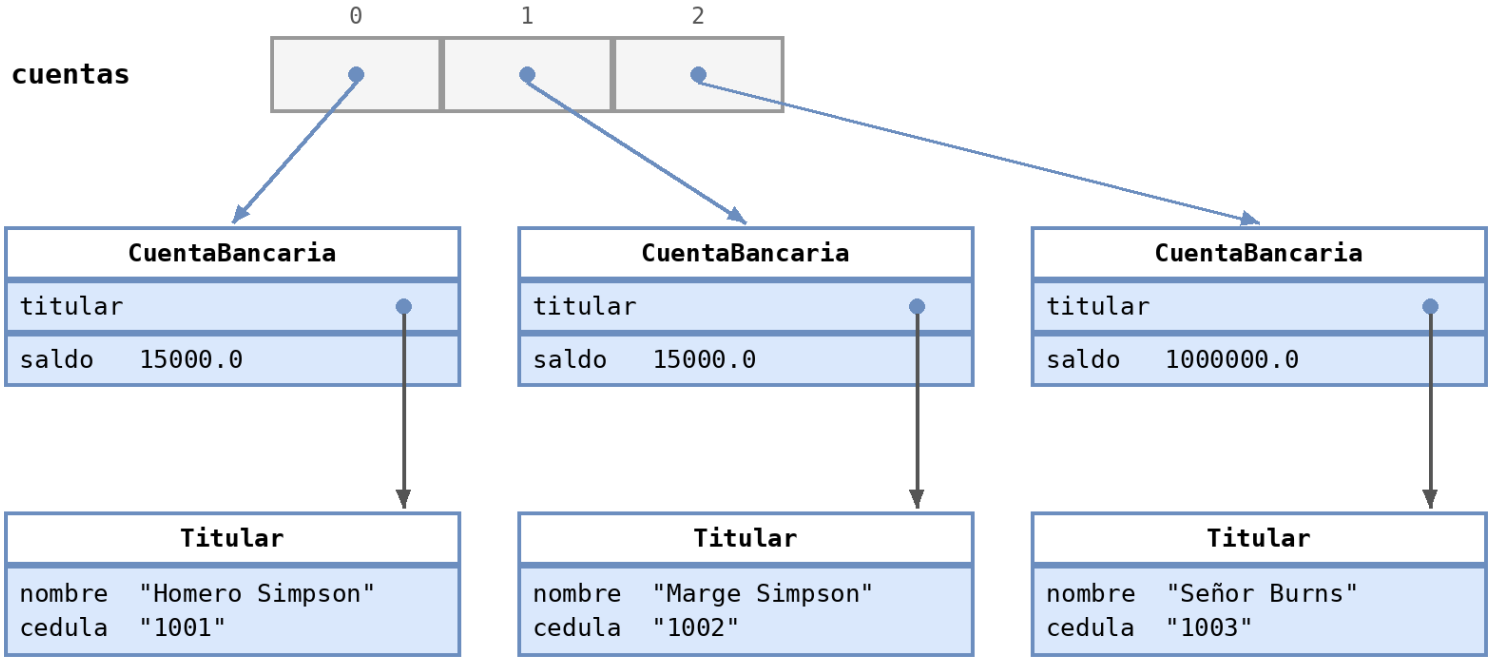
¿Por qué revisar los dos saldos? Una transferencia cambia dos cuentas: si solo se revisa una, un error en la otra pasa sin que nadie lo note.

¿Por qué revisar los saldos después del False? Para comprobar que el rechazo no movió dinero: ni Marge perdió ni Homero ganó.

Ejemplo 5: un arreglo de cuentas

Escriba un programa que guarde en un arreglo las cuentas de **Homero** (15000.0), **Marge** (15000.0) y el señor **Burns** (1000000.0), y que:

- Encuentre la posición de la cuenta con el mayor saldo y muestre su descripción.
- Le dé un bono de 10000.0 a cada cuenta y muestre todas las cuentas.



¿Para el escenario descrito cual seria la instrucción para obtener el salario mas alto?

Ejemplo 5: el mayor saldo

Para obtener el saldo más alto, se recorre el arreglo **`cuentas`** con el índice **`i`** y, en cada posición, se le pide el saldo al objeto **`CuentaBancaria`** con **`get_saldo()`**.

```
cuentas = [None] * 3
cuentas[0] = CuentaBancaria(Titular("Homero Simpson", "1001"), 15000.0)
cuentas[1] = CuentaBancaria(Titular("Marge Simpson", "1002"), 15000.0)
cuentas[2] = CuentaBancaria(Titular("Señor Burns", "1003"), 1000000.0)

pos_mayor = 0
for i in range(1, len(cuentas)):
    if cuentas[i].get_saldo() > cuentas[pos_mayor].get_saldo():
        pos_mayor = i
print(pos_mayor)
print(cuentas[pos_mayor])
```

```
2
Señor Burns: $1000000.0
```

Link simulación

- **¿Por qué se compara `cuentas[i].get_saldo()` y no `cuentas[i]`?** Porque se comparan saldos, no cuentas: el saldo es privado y se pide con su método.
- **¿El algoritmo es nuevo?** No, es la búsqueda de la posición del mayor que ya conocen. Lo único que cambia es que cada casilla guarda un objeto.
- **¿Qué hace `Titular(...)` dentro de `CuentaBancaria(...)`?** Crea el titular y se lo entrega a la cuenta en una sola línea.

Ejemplo 5: un bono para cada cuenta

Para dar el bono, se recorre el arreglo **`cuentas`** con el índice **`i`** y, en cada posición, se le pide al objeto **`CuentaBancaria`** que deposite con **`depositar(10000.0)`**.

```
for i in range(len(cuentas)):
    cuentas[i].depositar(10000.0)

for i in range(len(cuentas)):
    print(cuentas[i])
```

Link simulación

```
Homero Simpson: $25000.0
Marge Simpson: $25000.0
Señor Burns: $1010000.0
```

- **¿Por qué no se escribe `cuentas[i] = ...`?** Porque la casilla no cambia: sigue apuntando a la misma cuenta. Lo que cambia es el **`saldo`** dentro de esa cuenta, y eso lo hace **`depositar`**.
- **¿Y en un arreglo de números?** Ahí sí se reasigna la casilla: **`notas[i] = notas[i] + 0.5`**. Con objetos, se le pide al objeto que cambie.
- **¿Por qué `print(cuentas[i])` muestra un texto?** Porque **`print`** llama a **`__str__`**, como en el ventilador.

El sistema crece

- **Composición (◆)**: el banco es dueño de sus cuentas. Las crea, y si el banco desaparece, sus cuentas también. Una cuenta pertenece a un solo banco.
- **El rombo va del lado del todo (el banco), y cuentas es el nombre del atributo**: el arreglo que hasta ahora vivía suelto en el programa.
- **El titular no es parte del banco**: Homero sigue existiendo aunque el banco cierre. Por eso entre la cuenta y el titular hay una asociación, sin rombo.

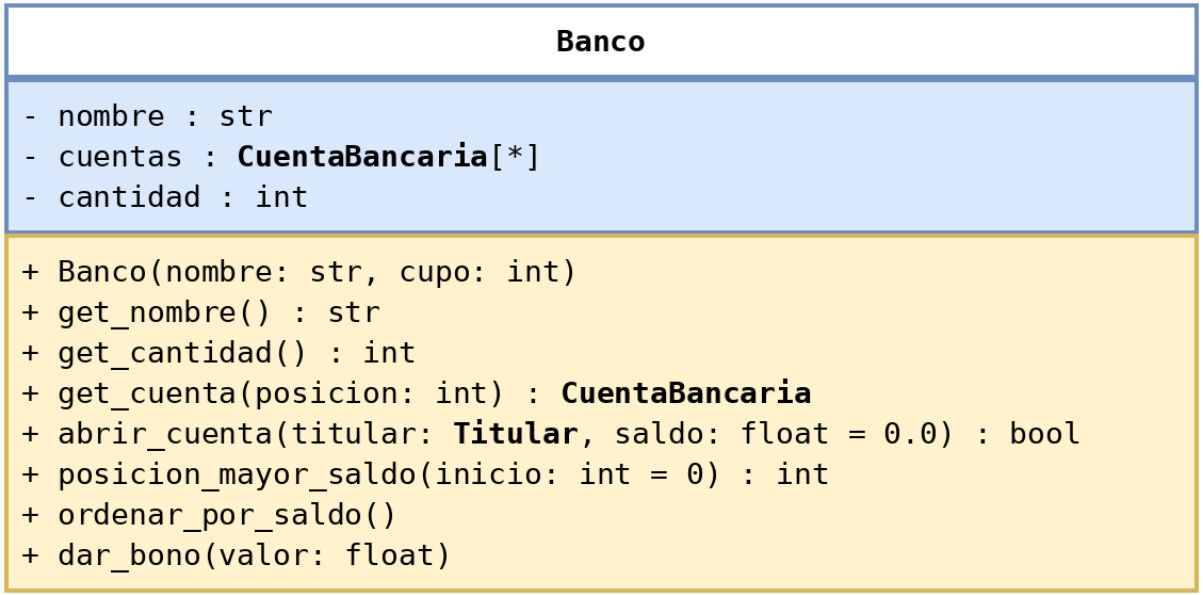


Precisión: la **composición** (◆) tiene un significado preciso en el estándar UML: el todo es responsable de que sus partes existan. La **agregación** (◇) es más ambigua, y algunos autores la evitan (Fowler la llama "un placebo de modelado"). Por eso, en un caso límite, el modelo depende del problema que se quiere resolver (link).

¿Cómo se escribe en Python una clase que guarda un arreglo de objetos? Esa es la pregunta del bloque 2, con un Curso que guarda Estudiantes (◇).

La clase Banco

- **`cuentas`**: **`CuentaBancaria[*]`** — un atributo que guarda muchas cuentas: el arreglo que en el Ejemplo 5 vivía suelto en el programa.
- **`abrir_cuenta`** crea la cuenta dentro del banco: por eso la relación es de composición (◆).



- **¿Qué significa [*]?** Que el atributo guarda muchos objetos. Es la misma multiplicidad **`0..*`** de la flecha del diagrama anterior, escrita dentro de la caja.
- **¿Por qué `abrir_cuenta` recibe un `Titular` y no una `CuentaBancaria`?** Porque el banco crea la cuenta; el titular, en cambio, ya existe fuera del banco.
- **¿Por qué `posicion_mayor_saldo` tiene `inicio = 0`?** Para usar el mismo método en dos casos: buscar el mayor de todo el banco y ordenar.

La clase Banco: atributos y métodos

Atributo	Tipo	Qué guarda
`nombre`	`str`	El nombre del banco
`cuentas`	`CuentaBancaria[*]`	El arreglo de cuentas, con tamaño cupo
`cantidad`	`int`	Cuántas cuentas hay abiertas (empieza en 0)

Método	Devuelve	Qué hace
`Banco(nombre, cupo)`	–	Crea el banco sin cuentas, con espacio para cupo cuentas
`get_nombre()`	`str`	Consulta el nombre
`get_cantidad()`	`int`	Consulta cuántas cuentas hay
`get_cuenta(posicion)`	`CuentaBancaria`	Consulta la cuenta de esa posición
`abrir_cuenta(titular, saldo)`	`bool`	Crea una cuenta en la siguiente posición libre; False si no hay cupo
`posicion_mayor_saldo(inicio)`	`int`	Busca la posición del mayor saldo desde inicio
`ordenar_por_saldo()`	–	Ordena las cuentas de mayor a menor saldo
`dar_bono(valor)`	–	Deposita valor en cada cuenta

`posicion_mayor_saldo` y **`dar_bono`** son los ciclos del **Ejemplo 5**, ahora dentro de la clase.

Banco: del UML a Python (1/2)

Clase Banco

```
class Banco:
    def __init__(self, nombre, cupo):
        self.__nombre = nombre
        self.__cuentas = [None] * cupo
        self.__cantidad = 0

    def get_nombre(self):
        return self.__nombre

    def get_cantidad(self):
        return self.__cantidad

    def get_cuenta(self, posicion):
        return self.__cuentas[posicion]

    def abrir_cuenta(self, titular, saldo=0.0):
        abierta = False
        if self.__cantidad < len(self.__cuentas):
            self.__cuentas[self.__cantidad] = CuentaBancaria(titular, saldo)
            self.__cantidad = self.__cantidad + 1
            abierta = True
        return abierta
```

- **¿Dónde quedó `cuentas = [None] * 3`?** Dentro del constructor: ahora es `self.__cuentas`, y el tamaño (**`cupo`**) lo decide quien crea el banco.
- **¿Qué hace abrir_cuenta?** Lo mismo que **`cuentas[0] = CuentaBancaria(...)`** del Ejemplo 5, pero en la siguiente posición libre, y solo si hay cupo.
- **¿Por qué `len(self.__cuentas)` y no `cupo`?** Porque cupo es un parámetro: deja de existir cuando el constructor termina. El arreglo sí recuerda su tamaño.

Banco: del UML a Python (2/2)

Clase Banco (continuación)

```
def posicion_mayor_saldo(self, inicio=0):
        pos_mayor = inicio
        for j in range(inicio + 1, self.__cantidad):
            if self.__cuentas[j].get_saldo() > self.__cuentas[pos_mayor].get_saldo():
                pos_mayor = j
        return pos_mayor

    def ordenar_por_saldo(self):
        for i in range(self.__cantidad - 1):
            pos_mayor = self.posicion_mayor_saldo(i)
            intercambiar(self.__cuentas, i, pos_mayor)

    def dar_bono(self, valor):
        for i in range(self.__cantidad):
            self.__cuentas[i].depositar(valor)
```

Función `intercambiar`

```
def intercambiar(lista, i, j):
    temp = lista[i]
    lista[i] = lista[j]
    lista[j] = temp
```

- **¿Por qué `range(..., self.__cantidad)` y no `len(...)`?** Porque solo se recorren las cuentas abiertas: las demás casillas siguen en **`None`**.
- **¿Qué hace `self.posicion_mayor_saldo(i)`?** El banco usa uno de sus propios métodos: con **`self.`** se llama a otro método del mismo objeto.
- **¿Cuántos intercambios hay por cada paso?** Uno: se intercambia la cuenta completa, con su titular y su saldo.

Banco: qué se va a probar

Las mismas pruebas del Ejemplo 5, y además lo nuevo: ordenar y el cupo del banco (5 cuentas).

#	Qué se hace	Resultado esperado	Por qué
1	Se abren las cuentas de Homero (15000.0), Marge (15000.0) y el señor Burns (1000000.0)	3 cuentas abiertas	Los mismos datos del Ejemplo 5
2	Se busca la posición del mayor saldo	2 (el señor Burns)	Igual que en el Ejemplo 5
3	Se da un bono de 10000.0 a cada cuenta	Homero queda con 25000.0	Igual que en el Ejemplo 5
4	Se ordenan las cuentas por saldo	El señor Burns queda en la posición 0	Nuevo: ordenar
5	Se abren las cuentas de Lisa y Bart	True las dos: el banco queda lleno (5 de 5)	Caso límite: se llena el cupo
6	Se intenta abrir la cuenta de Maggie	False: no hay cupo	Caso especial: arreglo lleno

Banco: las pruebas en Python

Cada **assert** corresponde a una fila de la tabla anterior

Código:

```
banco = Banco("Banco de Springfield", 5)
# 1. Se abren las tres cuentas
banco.abrir_cuenta(Titular("Homero Simpson", "1001"), 15000.0)
banco.abrir_cuenta(Titular("Marge Simpson", "1002"), 15000.0)
banco.abrir_cuenta(Titular("Señor Burns", "1003"), 1000000.0)
assert banco.get_cantidad() == 3
# 2. Mayor saldo
assert banco.posicion_mayor_saldo() == 2
# 3. Bono
banco.dar_bono(10000.0)
assert banco.get_cuenta(0).get_saldo() == 25000.0
# 4. Ordenar por saldo
banco.ordenar_por_saldo()
assert banco.get_cuenta(0).get_titular().get_nombre() == "Señor Burns"
# 5. Se llena el cupo
assert banco.abrir_cuenta(Titular("Lisa Simpson", "1004")) == True
assert banco.abrir_cuenta(Titular("Bart Simpson", "1005")) == True
# 6. Arreglo lleno
assert banco.abrir_cuenta(Titular("Maggie Simpson", "1006")) == False
print("Banco: todas las pruebas pasaron")
```

- **¿Por qué Maggie da False?** Porque el banco tiene cupo 5 y ya hay 5 cuentas: abrir_cuenta no se sale del arreglo.
- **¿Dónde quedaron los ciclos?** Dentro del banco: el programa solo pide; el banco sabe cómo hacerlo.

¿Por qué POO?

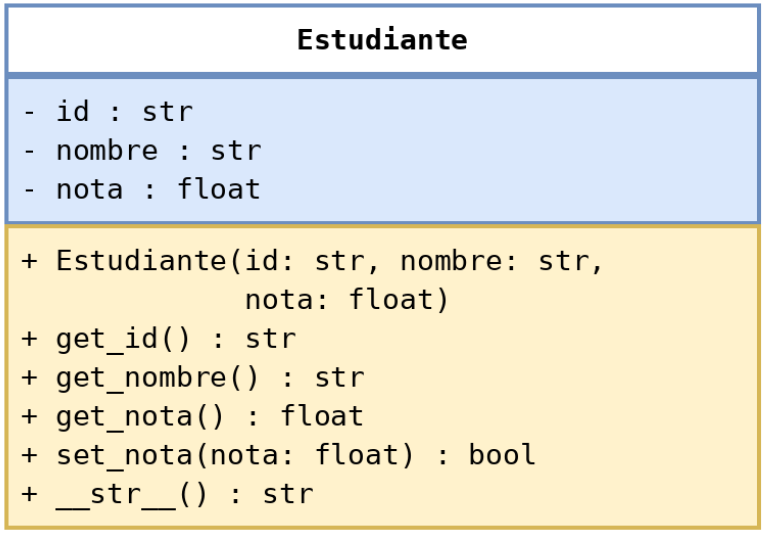
El tablero del inicio de la clase, ahora con el banco.

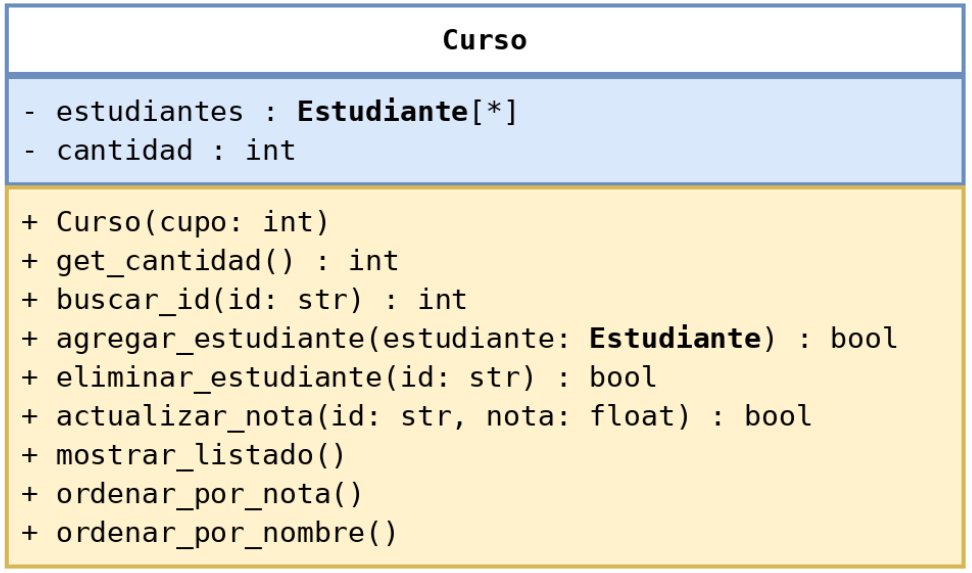
#	Con arreglos paralelos	Con objetos	Estado
1	Una cuenta no existe: nombres, cédulas y saldos sueltos	Existe: es una CuentaBancaria	✓ resuelta
2	Cada intercambio son tres (nombres, cédulas y saldos)	Se intercambia una cuenta	✓ resuelta
3	cantidad va a todas partes	Vive dentro de Banco	✓ resuelta
4	Ordenar por saldo y por nombre: dos funciones casi iguales	Dos métodos casi iguales	✗ no se resuelve
+	¿Y si se agrega el correo?	Solo cambia Titular	✓ resuelta

Con lo visto en esta clase, la fila 4 sigue igual; se resuelve pasando el criterio como parámetro, una idea fuera del alcance del curso. Lo demás lo resolvieron los objetos: datos juntos, un solo intercambio y la cantidad dentro del banco.

El bloque 2 en UML: Estudiante y Curso

El mismo problema de administración de notas, con el mismo menú, ahora con dos clases. Cada opción del menú es un método de Curso.





¿A qué se parece Curso? A **Banco**: un arreglo de objetos con cupo y cantidad. Lo que cambia es el tipo de objeto que guarda.

¿Por qué `agregar_estudiante` recibe un Estudiante y no sus datos? Porque el estudiante ya existe antes de entrar al curso. Es la diferencia con **`abrir_cuenta`**, y se verá en el diagrama siguiente.

Composición y agregación

Composición (◆): el banco crea sus cuentas. Si el banco desaparece, sus cuentas también. Una cuenta pertenece a un solo banco.




```
self.__cuentas[self.__cantidad] = CuentaBancaria(titular, saldo)
```

Agregación (◇): el curso recibe estudiantes que ya existen. Si el curso se cancela, los estudiantes siguen existiendo, y un estudiante puede estar en varios cursos.




```
self.__estudiantes[self.__cantidad] = estudiante
```

¿Cómo saber qué rombo va? Mirando el método que guarda el objeto:

- **`abrir_cuenta`** crea la cuenta adentro, con **`CuentaBancaria(titular, saldo)`**: composición (◆).
- **`agregar_estudiante`** recibe un estudiante que ya fue creado afuera y solo lo guarda: agregación (◇).

Errores típicos

La siguiente tabla muestra errores típicos al usar POO en Python:

Error	Lo que pasa	Corrección
`def calcular_area():`	`TypeError: Rectangulo.calcular_area() takes 0 positional arguments but 1 was given`	`def calcular_area(self):`
`return base * altura`	`NameError: name 'base' is not defined`	`return self.base * self.altura`
`print(r1.calcular_area)`	**No da error**: muestra `<bound method Rectangulo.calcular_area of ...>`	`print(r1.calcular_area())`
`r1 == r3`, con los mismos valores	Da `False`: si la clase no define cómo compararse, `==` funciona igual que `is`	Comparar los valores: `r1.get_base() == r3.get_base()`
Dos métodos con el mismo nombre	El segundo reemplaza al primero: `TypeError: ... missing 1 required positional argument`	Un solo método con valor por defecto, como el constructor del ventilador
`r1.__base = -5.0`, fuera de la clase	**No da error**: crea otro atributo aparte; la base verdadera no cambia	Usar el setter: `r1.set_base(5.0)`

Los errores más peligrosos son los que no dan error (en rojo): el programa sigue corriendo con un resultado equivocado.

Resumen

Concepto	En UML	En Python
Clase	Caja de tres partes: nombre, atributos, métodos	`class Rectangulo:`
Objeto	`r1 : Rectangulo` (subrayado)	`r1 = Rectangulo(3.0, 4.0)`
Constructor	`+ Rectangulo(base: float, altura: float)`	`def __init__(self, base, altura):`
Atributo privado	`- base : float`	`self.__base = base`
Método	`+ calcular_area() : float`	`def calcular_area(self):`
Método sin retorno	`+ encender()` (sin tipo)	Sin `return`: devuelve `None`
Getter / setter	`+ get_base() : float` / `+ set_base(base: float) : bool`	`return self.__base` / validar y asignar
Constante de clase	`+ LENTO : int = 1` (subrayada)	`LENTO = 1` dentro de la clase
Asociación	`CuentaBancaria ──> Titular`	Un atributo que guarda otro objeto
Composición	`Banco ◆── CuentaBancaria`	El método crea el objeto: `CuentaBancaria(...)`
Agregación	`Curso ◇── Estudiante`	El método guarda un objeto ya creado

La idea central: los datos y lo que se hace con ellos viven juntos, en el objeto.
Lo que sigue: construir en Python el Curso del diagrama, con el mismo menú de siempre.

Referencias

Para estudiar

- **Mera Banguero, C. A. Guía Teórica 02: Programación Orientada a Objetos**. Universidad de Antioquia.
- **CS50's Introduction to Programming with Python**, Lecture 8: Object-Oriented Programming. Harvard University. https://cs50.harvard.edu/python/weeks/8/
- **Documentación de Python**, Tutorial, sección 9.6 Variables privadas. https://docs.python.org/es/3.11/tutorial/classes.html#private-variables
- **Python Tutor** (visualización de objetos y referencias). https://pythontutor.com

Referencias

Libros

- **Liang, Y. D. Introduction to Programming Using Python**. Pearson. Capítulo "Objects and Classes" (estilo de los enunciados; ejercicio del ventilador).
- **Gaddis, T. Starting Out with Python. Pearson**. Capítulo "Classes and Object-Oriented Programming" (diagramas UML; ejemplo del celular).
- **Bloch, J. Effective Java**, 3.ª ed. Addison-Wesley, 2018. Item 49: "Check parameters for validity".
- **Meyer, B. Object-Oriented Software Construction**, 2.ª ed. Prentice Hall, 1997 (separación entre órdenes y consultas; diseño por contrato).

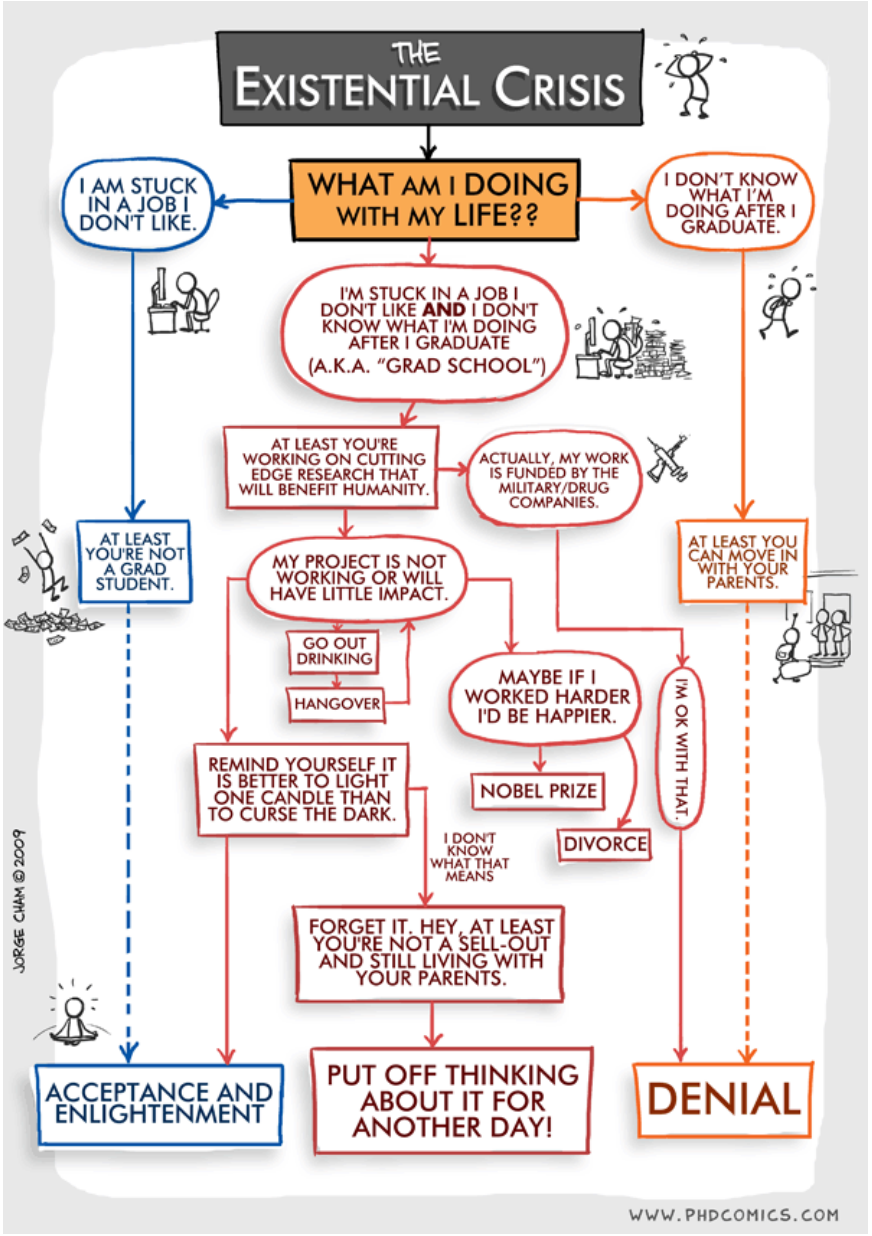
Referencias

UML

- **Object Management Group. Unified Modeling Language, versión 2.5.1**. https://www.omg.org/spec/UML/2.5.1
- **Kobryn, C. "UML 2001: A Standardization Odyssey"**. Communications of the ACM 42(10), 1999. https://cacm.acm.org/research/uml-2001
- **University of Edinburgh. Inf2-SEPP: Software Engineering Processes and Practices (diagramas de clases UML)**. https://opencourse.inf.ed.ac.uk/inf2-sepp
- **Fowler, M. "AggregationAndComposition" (2003).** https://martinfowler.com/bliki/AggregationAndComposition.html
- **Chua Hock Chuan. "Java Programming Tutorial: OOP — Composition, Inheritance & Polymorphism"**. Nanyang Technological University. https://www3.ntu.edu.sg/home/ehchua/programming/java/J3b_OOPInheritancePolymorphism.html

Eso es todo amigos





https://phdcomics.com/comics/archive.php?comicid=1431